

DATA 260 – HW 5

7- Community Sports League Fixtures

SJSU ID : 020053319 | SRI PRAJEET VILLYAL

Hardware: MacBook Pro, Apple M2, 8 GB RAM, macOS

Local model used: qwen2.5:3b (run locally through Ollama)

GitHub repository: <https://github.com/villyalsriprajeet-cmd/data260-3319>

Tagged commit hash:

Section 0 - Personal Configuration and Domain

Value	Definition
SID4	3319
PORT_BASE	8619
PREFIX	s3319
SEED	3319
VERIFY_SEED	263319
DOMAIN_ID	7

Part 1

1.Database

teams is the related entity and fixtures is the primary entity. Migration

003_hw05_relationship.sql changed the HW4 tables in place and kept all 200 teams and 5,000 fixtures.

Requirement	teams (related entity)	fixtures (primary entity)
Primary key	id INT AUTO_INCREMENT	id INT AUTO_INCREMENT
Primary / secondary field	name / city	fixture_title / venue
Unique field	contact_email (UNIQUE, valid email)	fixture_code (UNIQUE, FX-00001 format)
Numeric with default	–	tickets_available INT DEFAULT 500, CHECK >= 0
Foreign key	–	home_team_id NOT NULL → teams.id ON DELETE RESTRICT
Timestamps	created_at, updated_at	created_at, updated_at

Passwords are never stored as plain text. The users table keeps only a salted hash of each password. Each fixture is linked to its home team with ON DELETE RESTRICT, so MySQL will not delete a team that still has fixtures. The only automatic delete is for login sessions, so when a user is deleted, that user's sessions are deleted too.

```
code > api > sql > 003_hw05_relationship.sql
1  -- HW5 Part 1: teams become the related entity, fixtures get a code, ticket count, timestamps and a RESTRICT foreign key
2  USE s3319_rel;
3  -- teams: add the unique contact email and timestamps
4  ALTER TABLE teams
5      ADD COLUMN contact_email VARCHAR(255) NULL AFTER city,
6      ADD COLUMN created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
7      ADD COLUMN updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP;
8  -- give the 200 seeded teams an email built from their unique name
9  UPDATE teams SET contact_email = CONCAT(LOWER(REPLACE(name, ' ', '.')), '@s3319league.org') WHERE id > 0;
10 -- now every team has one, so make it required and unique
11 ALTER TABLE teams
12     MODIFY contact_email VARCHAR(255) NOT NULL,
13     ADD CONSTRAINT uq_teams_contact_email UNIQUE (contact_email);
14 -- fixtures: add the unique code, the ticket count with a default, and timestamps
15 ALTER TABLE fixtures
16     ADD COLUMN fixture_code VARCHAR(20) NULL AFTER fixture_title,
17     ADD COLUMN tickets_available INT NOT NULL DEFAULT 500 AFTER venue,
18     ADD COLUMN created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
19     ADD COLUMN updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP;
20 -- give the 5,000 seeded fixtures a code like FX-00001 and a repeatable ticket count between 0 and 500
21 UPDATE fixtures SET fixture_code = CONCAT('FX-', LPAD(id, 5, '0')), tickets_available = MOD(id * 37, 501) WHERE id > 0;
22 -- drop the old SET NULL foreign key so it can be recreated as RESTRICT
23 ALTER TABLE fixtures DROP FOREIGN KEY fk_fixtures_team;
24 -- every fixture must have a home team, tickets never negative
25 ALTER TABLE fixtures
26     MODIFY fixture_code VARCHAR(20) NOT NULL,
27     MODIFY home_team_id INT NOT NULL,
28     ADD CONSTRAINT uq_fixtures_code UNIQUE (fixture_code),
29     ADD CONSTRAINT chk_fixtures_tickets CHECK (tickets_available >= 0),
30     ADD CONSTRAINT fk_fixtures_team FOREIGN KEY (home_team_id) REFERENCES teams(id) ON DELETE RESTRICT;
```

```
(base) villyalsirprajeet@Villyals-MacBook-Pro data260-3319 % mysql -u root -p < code/api/sql/003_hw05_relationship.sql
Enter password:
(base) villyalsirprajeet@Villyals-MacBook-Pro data260-3319 % mysql -u root -p s3319_rel -e "DESCRIBE teams; DESCRIBE fixtures; SELECT id,name,city,contact_email,created_at FROM teams LIMIT 3; SELECT id,fixture_title,fixture_code, tickets_available,home_team_id,created_at FROM fixtures LIMIT 3;"
Enter password:
+----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+----+-----+-----+-----+-----+-----+
| id | int | NO | PRI | NULL | auto_increment |
| name | varchar(100) | NO | | NULL | |
| city | varchar(100) | NO | | NULL | |
| contact_email | varchar(255) | NO | UNI | NULL | |
| created_at | datetime | NO | | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
| updated_at | datetime | NO | | CURRENT_TIMESTAMP | DEFAULT_GENERATED on update CURRENT_TIMESTAMP |
+----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+----+-----+-----+-----+-----+-----+
| id | int | NO | PRI | NULL | auto_increment |
| fixture_title | varchar(200) | NO | | NULL | |
| fixture_code | varchar(20) | NO | UNI | NULL | |
| venue | varchar(200) | NO | MUL | NULL | |
| tickets_available | int | NO | | 500 | |
| home_team_id | int | NO | MUL | NULL | |
| created_at | datetime | NO | | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
| updated_at | datetime | NO | | CURRENT_TIMESTAMP | DEFAULT_GENERATED on update CURRENT_TIMESTAMP |
+----+-----+-----+-----+-----+-----+
| id | name | city | contact_email | created_at |
+----+-----+-----+-----+-----+
| 1 | Pleasanton Hawks | Pleasanton | pleasanton.hawks@s3319league.org | 2026-10-01 17:14:36 |
| 2 | Santa Clara Hawks | Santa Clara | santa.clara.hawks@s3319league.org | 2026-10-01 17:14:36 |
| 3 | Pleasanton Bears | Pleasanton | pleasanton.bears@s3319league.org | 2026-10-01 17:14:36 |
+----+-----+-----+-----+-----+
| id | fixture_title | fixture_code | tickets_available | home_team_id | created_at |
+----+-----+-----+-----+-----+
| 1 | Pleasanton Hawks vs Cupertino Spartans - Matchweek 32 | FX-00001 | 37 | 1 | 2026-10-01 17:14:36 |
| 2 | Santa Clara Hawks vs Fremont Sharks - Matchweek 32 | FX-00002 | 74 | 2 | 2026-10-01 17:14:36 |
| 3 | Pleasanton Bears vs Campbell Tigers - Matchweek 17 | FX-00003 | 111 | 3 | 2026-10-01 17:14:36 |
+----+-----+-----+-----+-----+

```

```
(base) villyalsirprajeet@Villyals-MacBook-Pro data260-3319 % mysql -u root -p -e "SELECT CONSTRAINT_NAME, TABLE_NAME, REFERENCED_TABLE_NAME, DELETE_RULE FROM information_schema.REFERENTIAL_CONSTRAINTS WHERE CONSTRAINT_SCHEMA='s3319_rel'; SELECT id, email, LEFT(password_hash, 45) AS password_hash_start FROM s3319_rel.users;"
Enter password:
+-----+-----+-----+-----+
| CONSTRAINT_NAME | TABLE_NAME | REFERENCED_TABLE_NAME | DELETE_RULE |
+-----+-----+-----+-----+
| fk_sessions_user | sessions | users | CASCADE |
| fk_fixtures_team | fixtures | teams | RESTRICT |
+-----+-----+-----+-----+
| id | email | password_hash_start |
+----+-----+-----+
| 1 | prajeet@s3319.com | 821c764aab958a951a22167029d7dfa56340496d9443 |
+----+-----+-----+
(base) villyalsirprajeet@Villyals-MacBook-Pro data260-3319 %
```

Constraint check output

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL TRUFFLE JUPYTER
(base) villyalsirprajeet@Villyals-MacBook-Pro data260-3319 % mysql -u root -p s3319_rel -e "DELETE FROM teams WHERE id=1;"
Enter password:
ERROR 1451 (23000) at line 1: Cannot delete or update a parent row: a foreign key constraint fails ('s3319_rel', 'fixtures', CONSTRAINT 'fk_fixtures_team' FOREIGN KEY ('home_team_id') REFERENCES 'teams' ('id') ON DELETE RESTRICT)
(base) villyalsirprajeet@Villyals-MacBook-Pro data260-3319 % mysql -u root -p s3319_rel -e "INSERT INTO fixtures (fixture_title,fixture_code,venue,home_team_id) VALUES ('x','FX-00001','v',1);"
Enter password:
ERROR 1062 (23000) at line 1: Duplicate entry 'FX-00001' for key 'fixtures.uq_fixtures_code'
(base) villyalsirprajeet@Villyals-MacBook-Pro data260-3319 % mysql -u root -p s3319_rel -e "UPDATE fixtures SET tickets_available=1 WHERE id=1;"
Enter password:
ERROR 3819 (HY000) at line 1: Check constraint 'chk_fixtures_tickets' is violated.
(base) villyalsirprajeet@Villyals-MacBook-Pro data260-3319 %
```

2. API Implementation

The HW4 FastAPI service was extended with a teams router and an updated fixtures router. Every route still needs a logged in session.

Pydantic validation (code/api/schemas.py)

```
CODE_PATTERN = r"^[FX-]\d{5}$" # fixture code format
class TeamCreate(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True) # trim spaces before checking
    name: str = Field(min_length=1, max_length=100)
    city: str = Field(min_length=1, max_length=100)
    contact_email: EmailStr # must be a valid email address

class FixtureCreate(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True)
    fixture_title: str = Field(min_length=1, max_length=200)
    fixture_code: str = Field(pattern=CODE_PATTERN) # rejects anything not like FX-00001
    venue: str = Field(min_length=1, max_length=200)
    tickets_available: int = Field(default=500, ge=0) # default 500, never negative
    home_team_id: int = Field(gt=0)
```

409 handling, pagination, RESTRICT delete, relationship query (code/api/routers/teams.py)

```
def commit_or_409(db: Session):
    try:
        db.commit()
    except IntegrityError:
        db.rollback() # undo the failed write
        raise HTTPException(status_code=409, detail="contact_email already used by another team")

@router.post("", response_model=TeamOut, status_code=201)
def create_team(payload: TeamCreate, db: Session = Depends(get_db)):
    team = Team(**payload.model_dump()) # id and timestamps come from MySQL
    db.add(team)
    commit_or_409(db)
    db.refresh(team) # reload to get id and timestamps
    return team

@router.get("", response_model=list[TeamOut])
def list_teams(skip: int = Query(0, ge=0), limit: int = Query(20, ge=1, le=100), db: Session = Depends(get_db)):
    return db.query(Team).order_by(Team.id).offset(skip).limit(limit).all() # one page of teams

@router.get("/{team_id}", response_model=TeamOut)
def get_team(team_id: int, db: Session = Depends(get_db)):
    return get_team_or_404(db, team_id)

@router.put("/{team_id}", response_model=TeamOut)
def update_team(team_id: int, payload: TeamUpdate, db: Session = Depends(get_db)):
    team = get_team_or_404(db, team_id)
    for field, value in payload.model_dump(exclude_unset=True, exclude_none=True).items():
        setattr(team, field, value) # change only the fields that were sent
    commit_or_409(db)
    db.refresh(team)
    return team

@router.delete("/{team_id}")
def delete_team(team_id: int, db: Session = Depends(get_db)):
    team = get_team_or_404(db, team_id)
    count = db.query(func.count(Fixture.id)).filter(Fixture.home_team_id == team_id).scalar() # fixtures still linked
    if count:
        raise HTTPException(status_code=409, detail=f"Cannot delete team {team_id}: it still has {count} fixture(s)")
    db.delete(team)
    db.commit()
    return {"message": "Team deleted", "id": team_id}

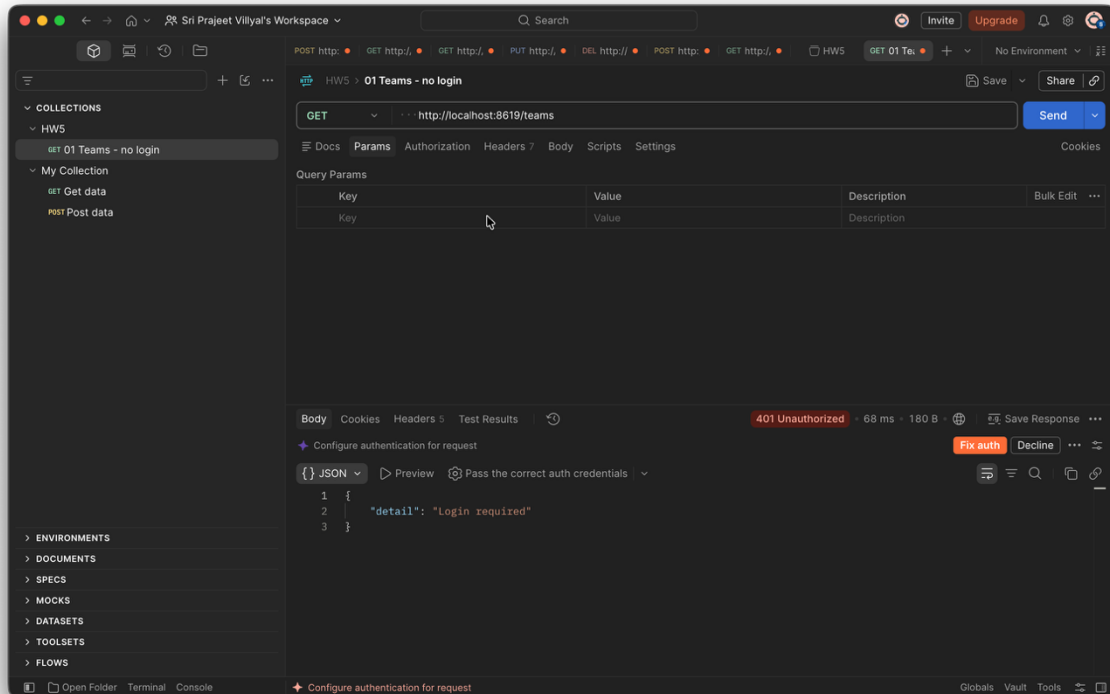
@router.get("/{team_id}/fixtures", response_model=list[FixtureOut])
def fixtures_for_team(team_id: int, skip: int = Query(0, ge=0), limit: int = Query(50, ge=1, le=100),
    db: Session = Depends(get_db)):
    get_team_or_404(db, team_id) # 404 if the team does not exist
    return (db.query(Fixture).filter(Fixture.home_team_id == team_id) # relationship query
        .order_by(Fixture.id).offset(skip).limit(limit).all())
```

Create and partial update (code/api/routers/fixtures.py)

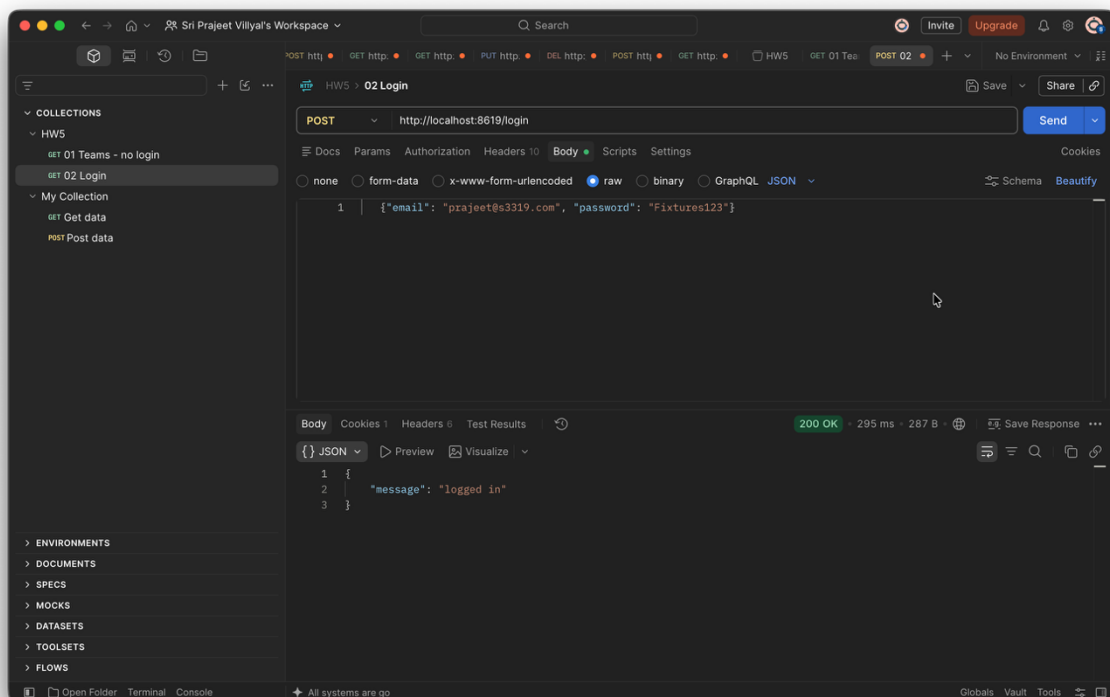
```
code > api > routers > fixtures.py > delete_fixture
1 # HWS fixture CRUD routes (code, tickets and home team added), every route requires a logged in session
2 from fastapi import APIRouter, Depends, HTTPException, Query
3 from sqlalchemy.exc import IntegrityError
4 from sqlalchemy.orm import Session, joinedload
5 from ..database import get_db
6 from ..models import Fixture, Team
7 from ..schemas import FixtureCreate, FixtureOut, FixtureUpdate, FixtureWithTeam
8 from ..security import require_session
9 router = APIRouter(prefix="/fixtures", tags=["Fixtures"], dependencies=[Depends(require_session)]) # 401 unless logged in
10 def get_fixture_or_404(db: Session, fixture_id: int) -> Fixture:
11     fixture = db.get(Fixture, fixture_id) # look up by primary key
12     if fixture is None:
13         raise HTTPException(status_code=404, detail=f"Fixture {fixture_id} not found")
14     return fixture
15 def check_team_exists(db: Session, team_id: int):
16     if db.get(Team, team_id) is None:
17         raise HTTPException(status_code=404, detail=f"Team {team_id} not found for home_team_id")
18 def commit_or_409(db: Session):
19     try:
20         db.commit()
21     except IntegrityError:
22         db.rollback() # undo the failed write
23         raise HTTPException(status_code=409, detail="fixture_code already used by another fixture")
24 @router.post("", response_model=FixtureOut, status_code=201)
25 def create_fixture(payload: FixtureCreate, db: Session = Depends(get_db)):
26     check_team_exists(db, payload.home_team_id)
27     fixture = Fixture(**payload.model_dump()) # id and timestamps come from MySQL
28     db.add(fixture)
29     commit_or_409(db)
30     db.refresh(fixture) # reload to get id and timestamps
31     return fixture
32 @router.get("", response_model=list[FixtureOut])
33 def list_fixtures(skip: int = Query(0, ge=0), limit: int = Query(20, ge=1, le=100), db: Session = Depends(get_db)):
34     return db.query(Fixture).order_by(Fixture.id.desc()).offset(skip).limit(limit).all() # newest first, one page
35 @router.get("/naive", response_model=list[FixtureWithTeam])
36 def list_fixtures_naive(limit: int = Query(10, ge=1, le=500), db: Session = Depends(get_db)):
37     fixtures = db.query(Fixture).order_by(Fixture.id).limit(limit).all() # 1 query for the page
38     for f in fixtures:
39         _ = f.home_team # lazy load: 1 extra query per fixture (N+1)
40     return fixtures
41 @router.get("/fixed", response_model=list[FixtureWithTeam])
42 def list_fixtures_fixed(limit: int = Query(10, ge=1, le=500), db: Session = Depends(get_db)):
43     return (db.query(Fixture).options(joinedload(Fixture.home_team)).order_by(Fixture.id).limit(limit).all()) # teams come in the same query via a JOIN
44 @router.get("/{fixture_id}", response_model=FixtureOut)
45 def get_fixture(fixture_id: int, db: Session = Depends(get_db)):
46     return get_fixture_or_404(db, fixture_id)
47 @router.put("/{fixture_id}", response_model=FixtureOut)
48 def update_fixture(fixture_id: int, payload: FixtureUpdate, db: Session = Depends(get_db)):
49     fixture = get_fixture_or_404(db, fixture_id)
50     changes = payload.model_dump(exclude_unset=True, exclude_none=True) # only the fields that were sent
51     if "home_team_id" in changes:
52         check_team_exists(db, changes["home_team_id"])
53     for field, value in changes.items():
54         setattr(fixture, field, value)
55     commit_or_409(db)
56     db.refresh(fixture)
57     return fixture
58 @router.delete("/{fixture_id}")
59 def delete_fixture(fixture_id: int, db: Session = Depends(get_db)):
60     fixture = get_fixture_or_404(db, fixture_id)
61     db.delete(fixture) # remove the row from MySQL
62     db.commit()
63     return {"message": "Fixture deleted", "id": fixture_id}
```

Postman Outputs

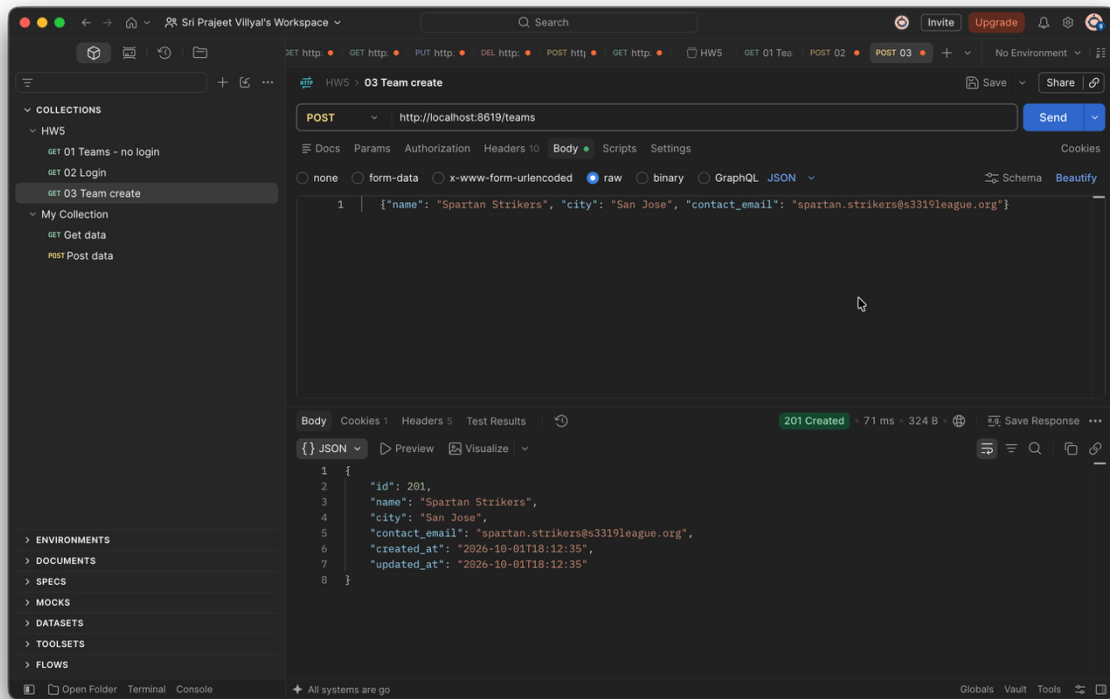
GET /teams before login - 401 Login required



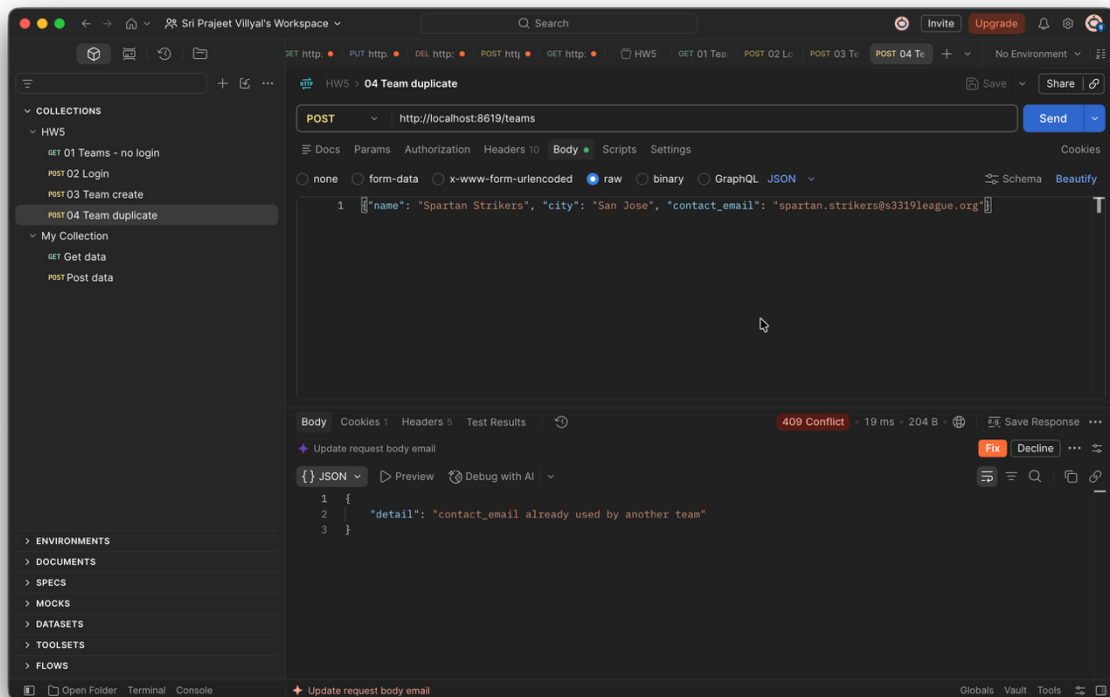
POST /login - 200 (HttpOnly session cookie set)



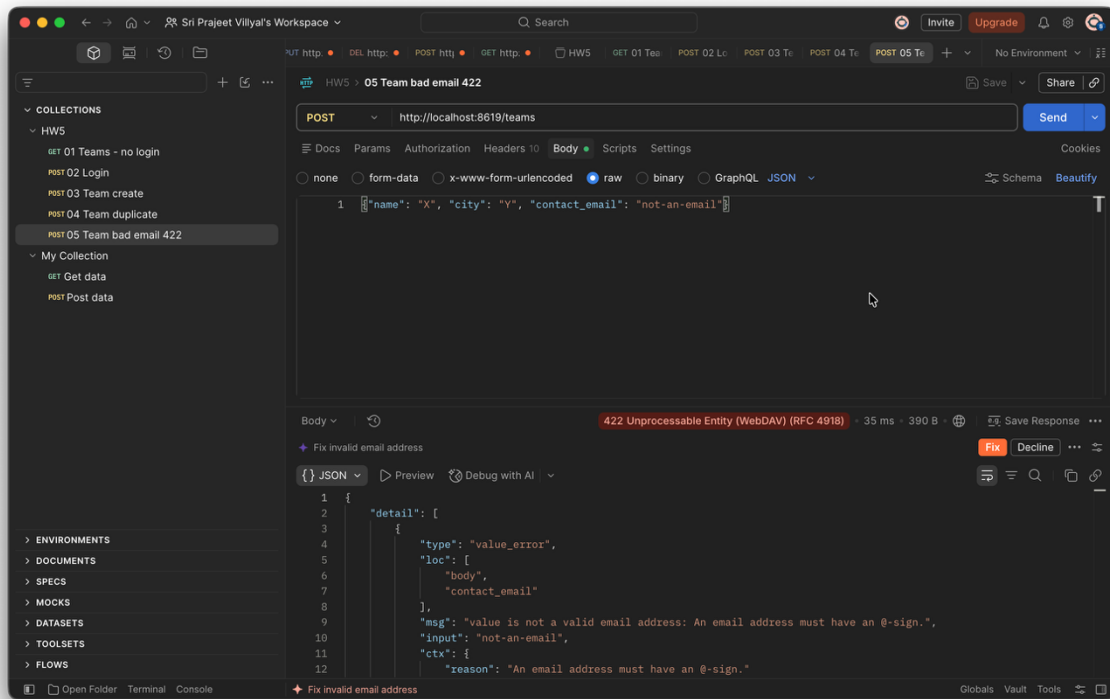
POST /teams - 201



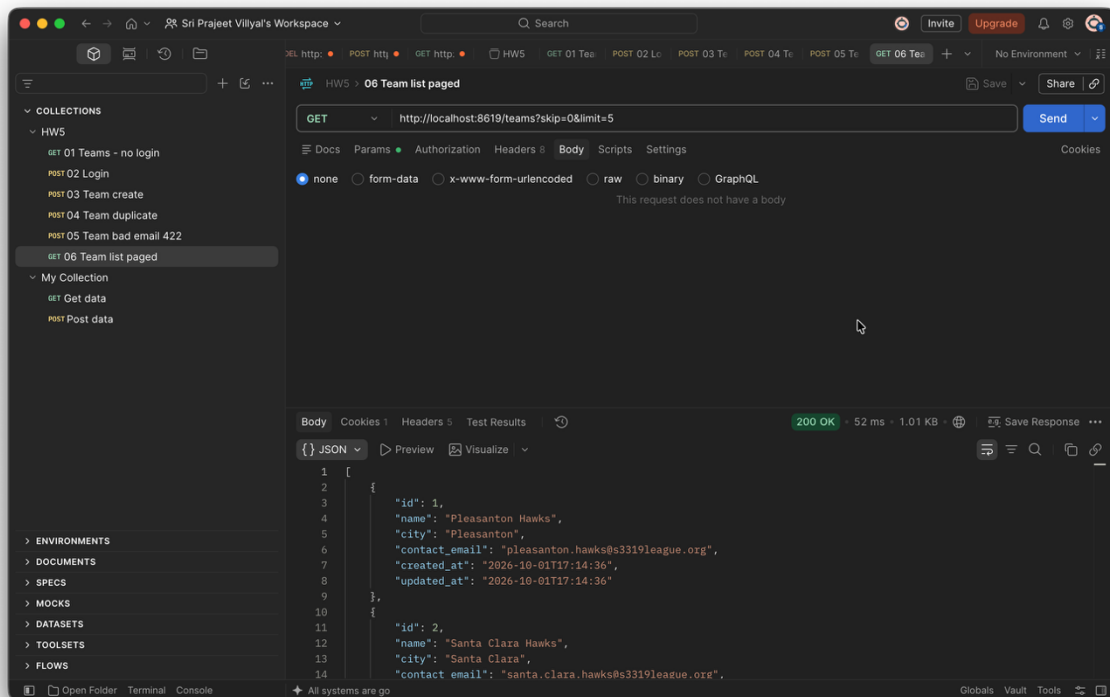
POST /teams same email - 409 Conflict



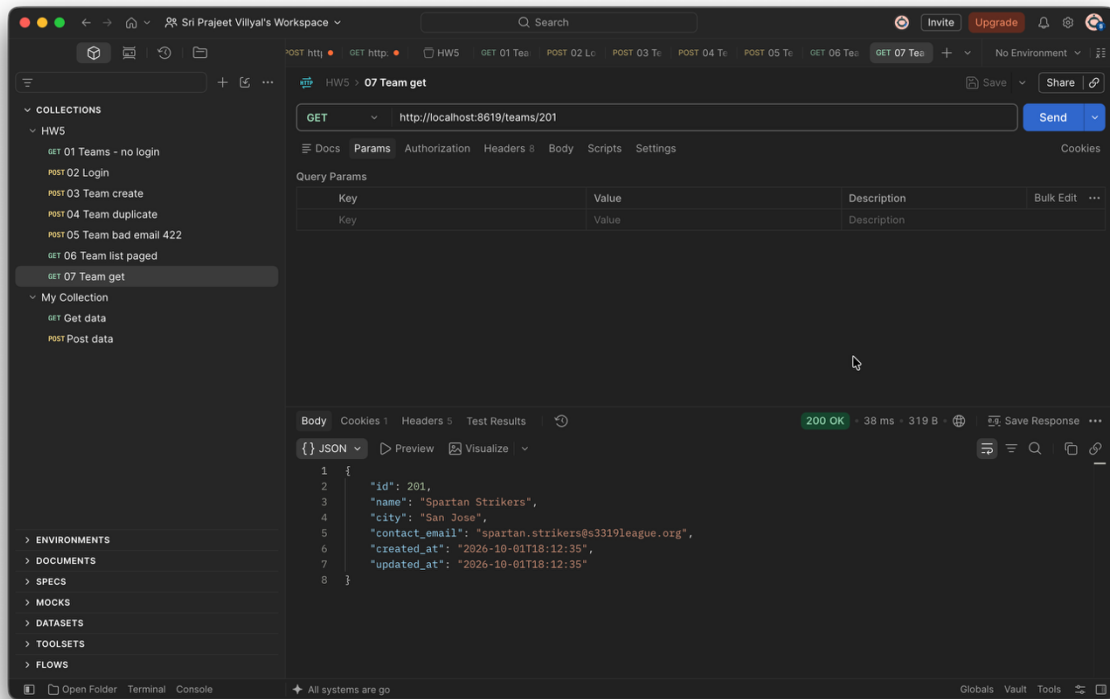
POST /teams invalid email - 422



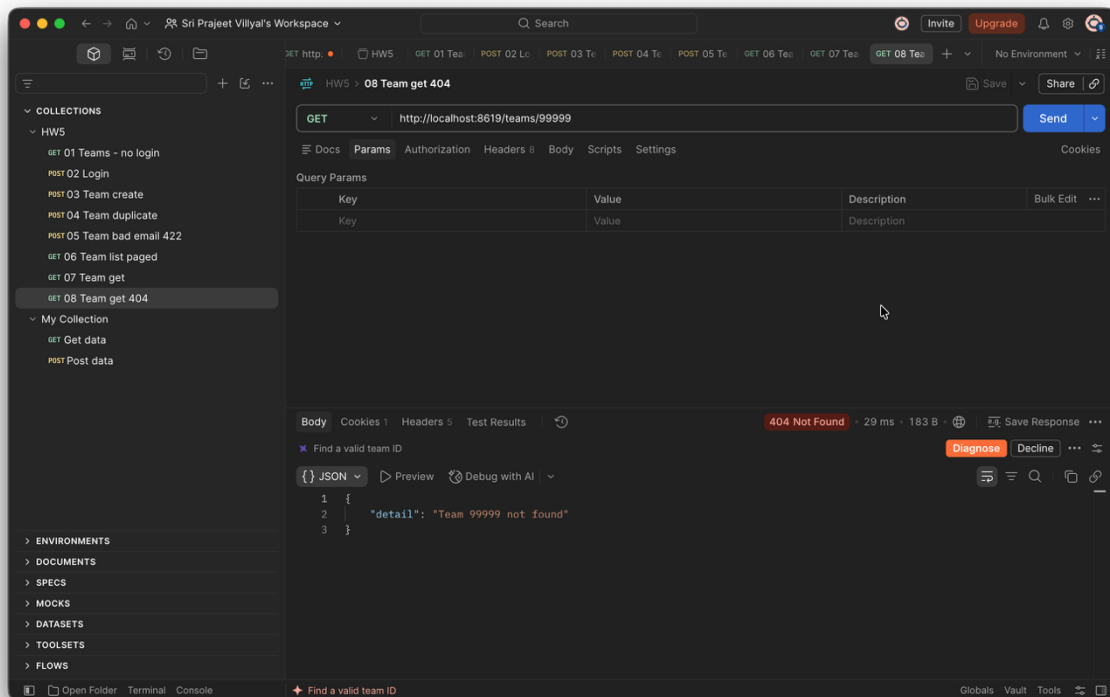
Pagination



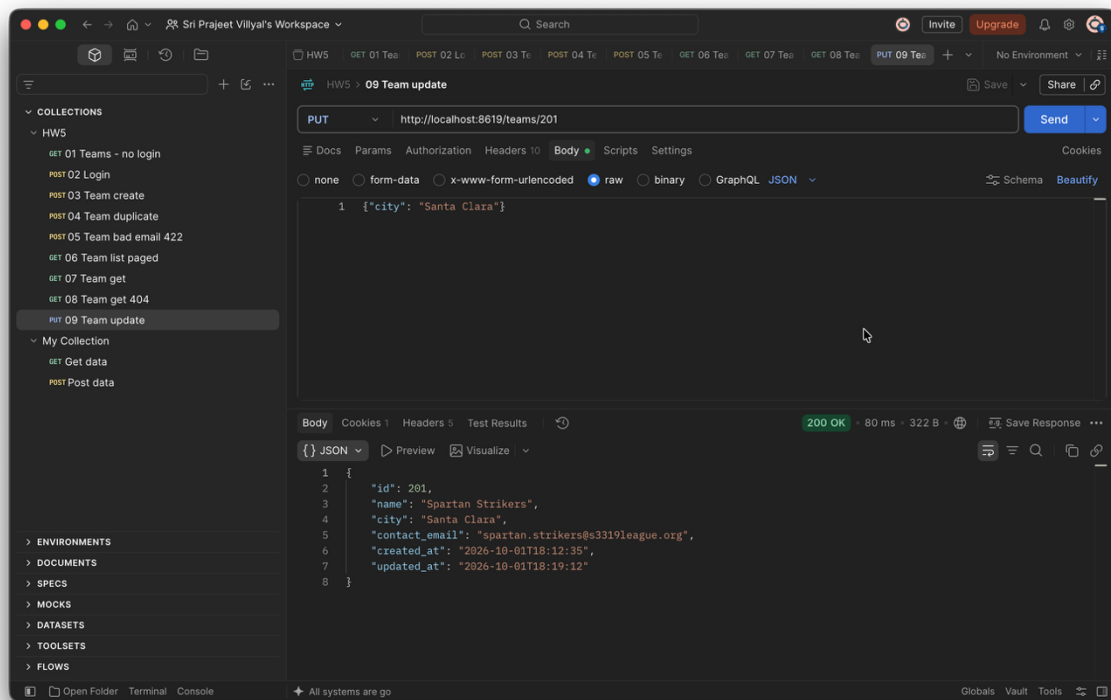
GET /teams/201 - 200



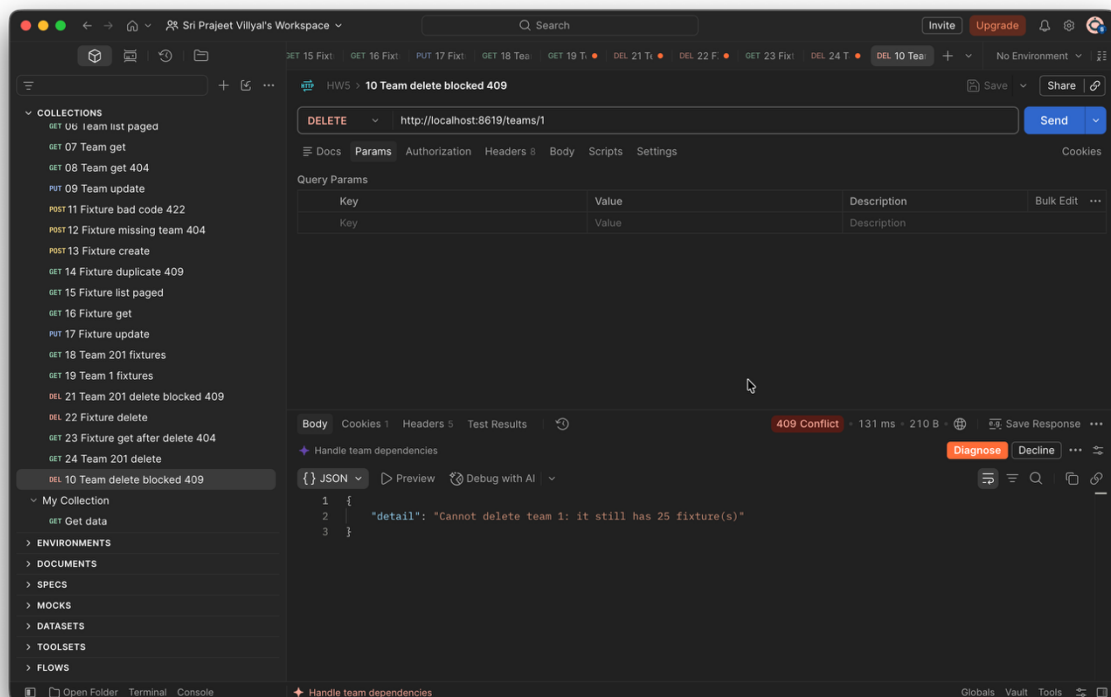
GET /teams/99999 - 404



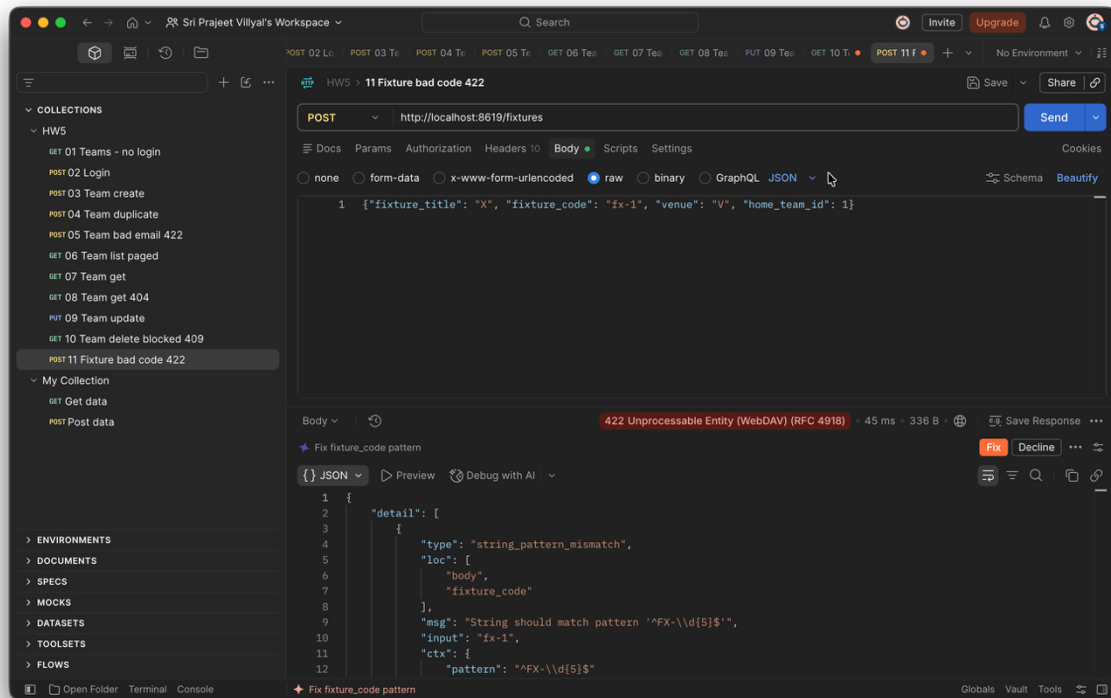
PUT /teams/201 {city} - 200, updated_at changed



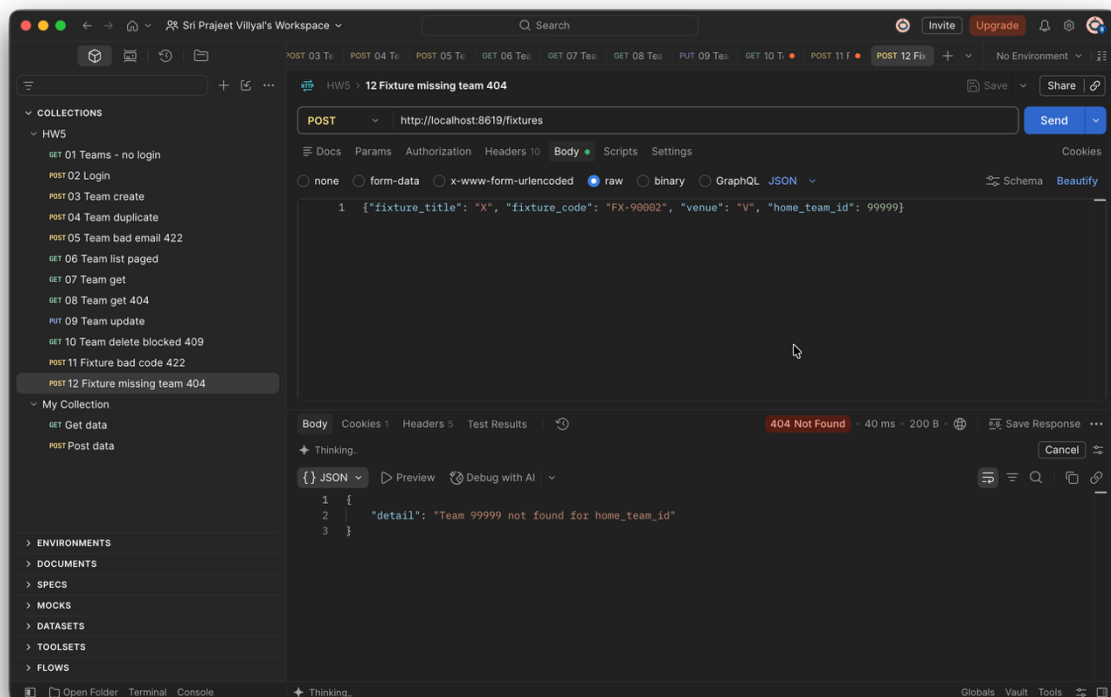
DELETE /teams/1 - 409, team still has 25 fixtures



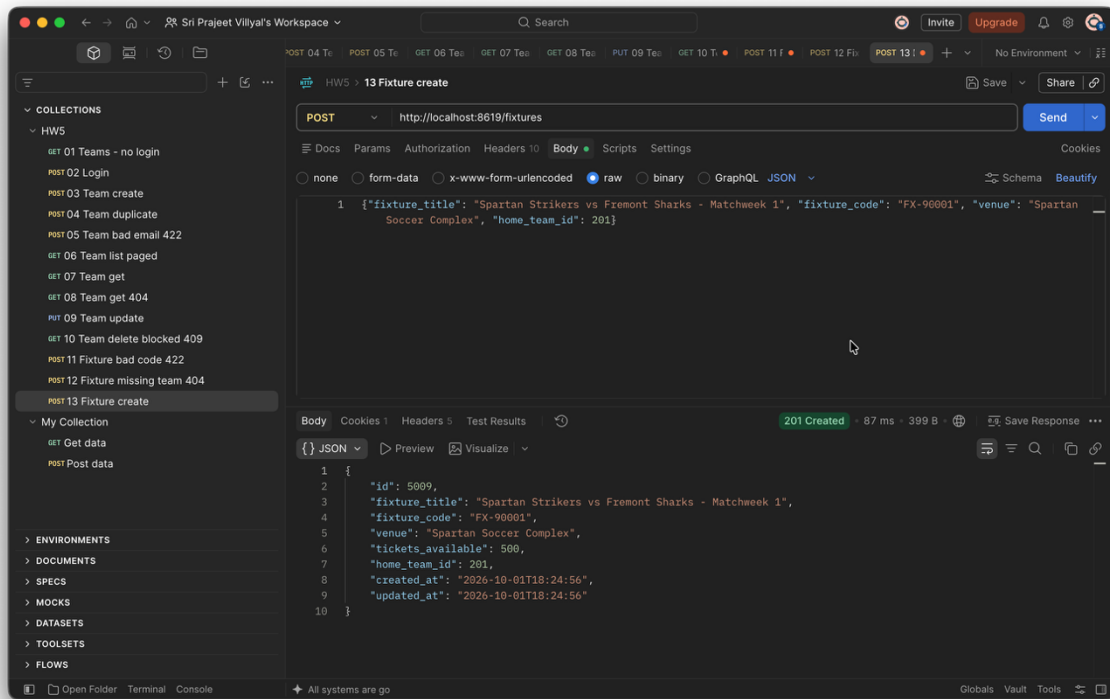
422 pattern mismatch



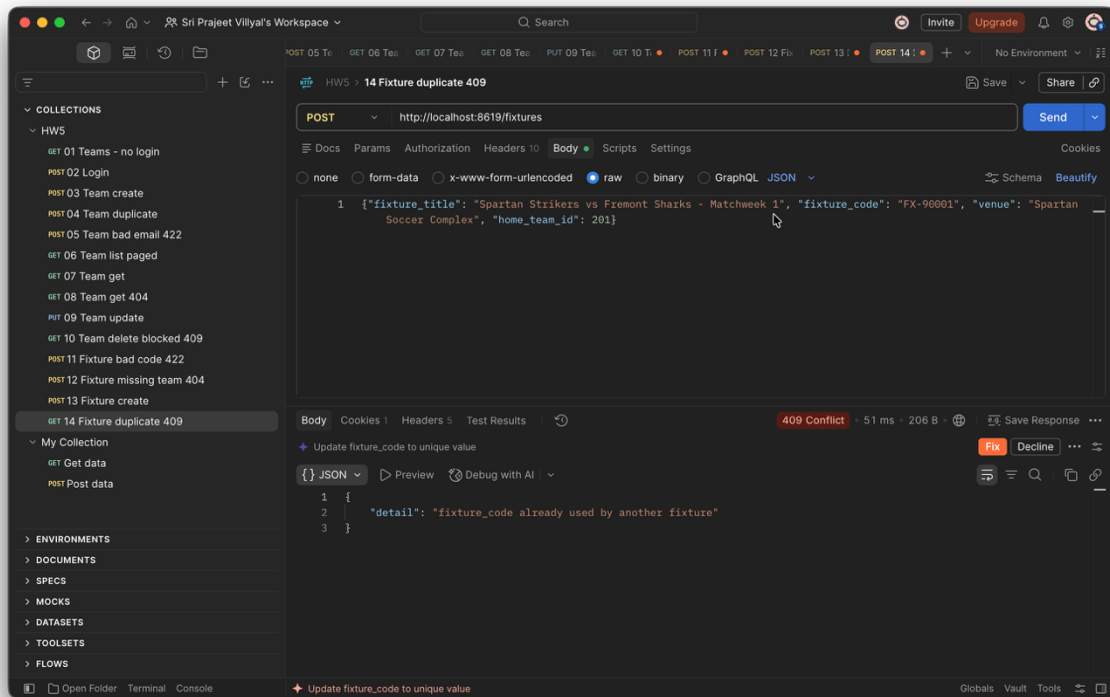
POST /fixtures home_team_id 99999 - 404



POST /fixtures - 201, tickets_available defaults to 500



POST /fixtures same code - 409 Conflict



Fixture list paged, newest first

The screenshot shows a Postman interface with a collection named 'HW5'. The selected request is 'GET 15 Fixture list paged' with the URL `http://localhost:8619/fixtures?skip=0&limit=5`. The response is a 200 OK status with a 130 ms latency and 1.36 KB body. The response body is a JSON array of two fixture objects.

Key	Value	Description	Bulk Edit
skip	0		
limit	5		

```
1 [
2   {
3     "id": 5009,
4     "fixture_title": "Spartan Strikers vs Fremont Sharks - Matchweek 1",
5     "fixture_code": "FX-90001",
6     "venue": "Spartan Soccer Complex",
7     "tickets_available": 500,
8     "home_team_id": 201,
9     "created_at": "2026-10-01T18:24:56",
10    "updated_at": "2026-10-01T18:24:56"
11  },
12  {
13    "id": 5000,
14    "fixture_title": "San Mateo Sharks vs Cupertino Hawks - Matchweek 3",
15  }
16 ]
```

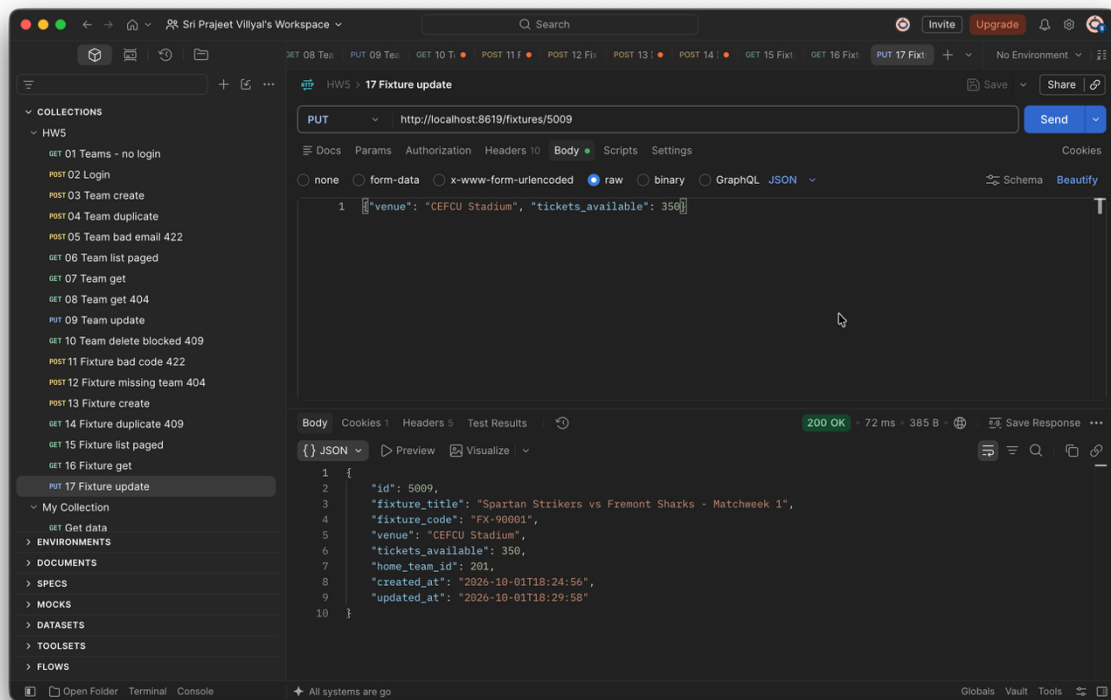
GET /fixtures/5009 - 200

The screenshot shows a Postman interface with a collection named 'HW5'. The selected request is 'GET 16 Fixture get' with the URL `http://localhost:8619/fixtures/5009`. The response is a 200 OK status with a 157 ms latency and 394 B body. The response body is a JSON object representing a single fixture.

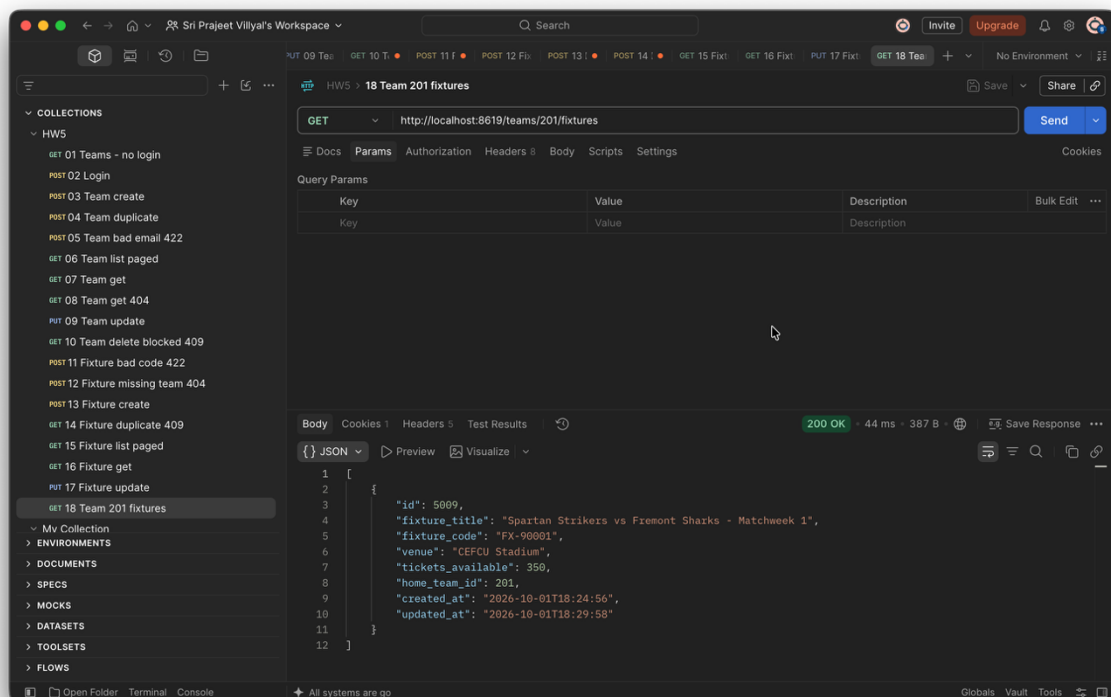
Key	Value	Description	Bulk Edit

```
1 {
2   "id": 5009,
3   "fixture_title": "Spartan Strikers vs Fremont Sharks - Matchweek 1",
4   "fixture_code": "FX-90001",
5   "venue": "Spartan Soccer Complex",
6   "tickets_available": 500,
7   "home_team_id": 201,
8   "created_at": "2026-10-01T18:24:56",
9   "updated_at": "2026-10-01T18:24:56"
10 }
```

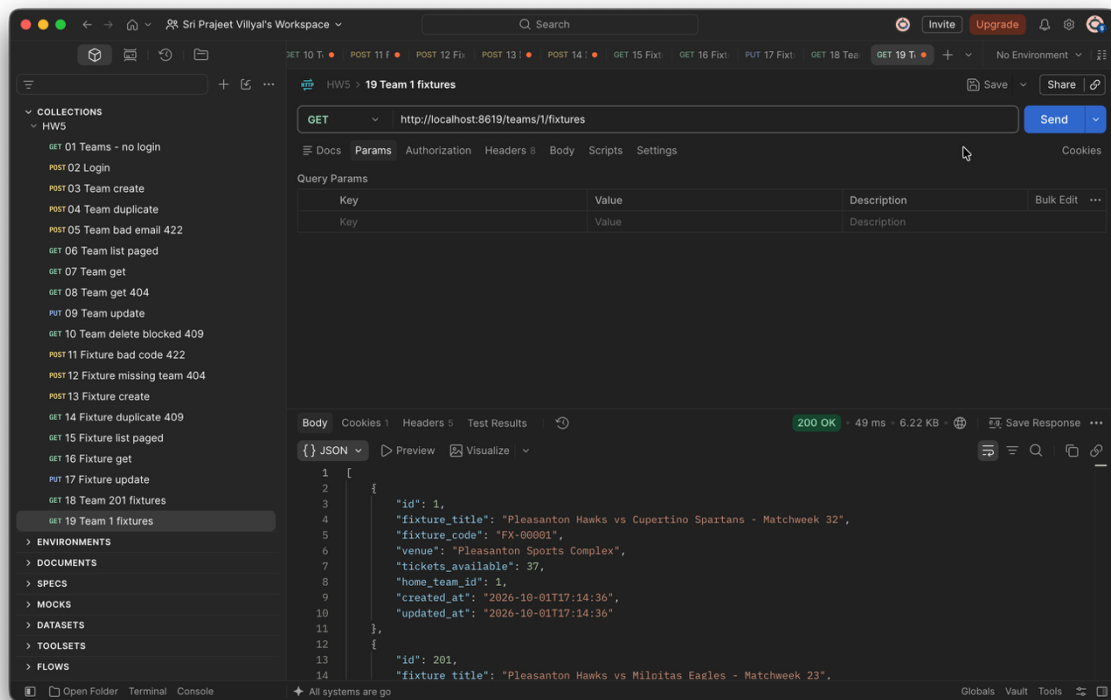
PUT /fixtures/5009 {venue, tickets} - 200



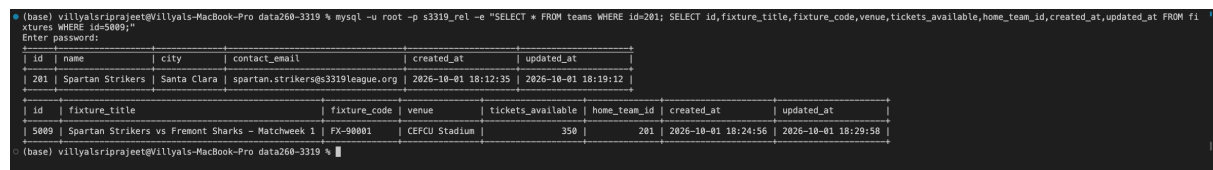
GET /teams/201/fixtures - 200 (relationship query)



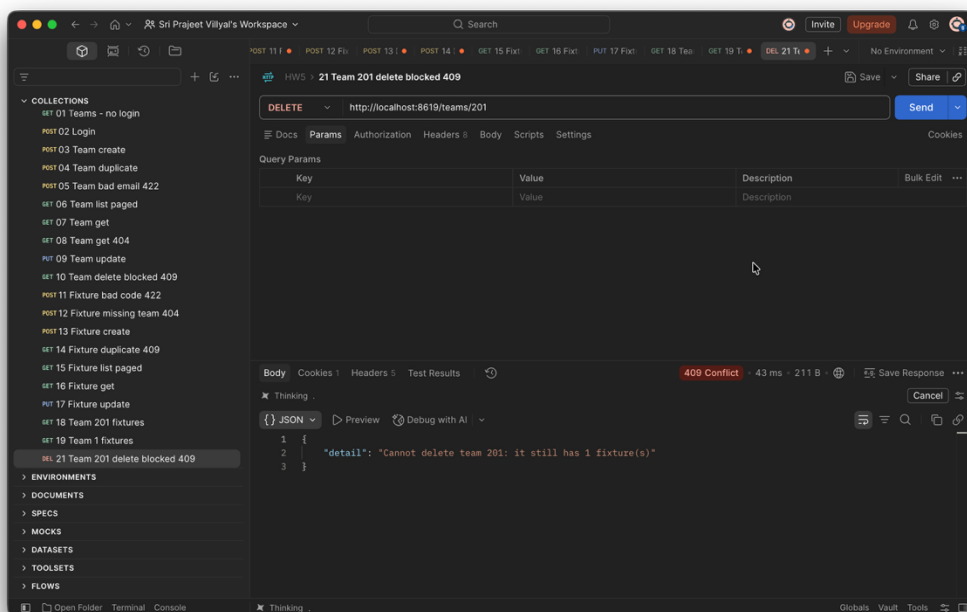
GET /teams/1/fixtures - 200, team 1's 25 fixtures



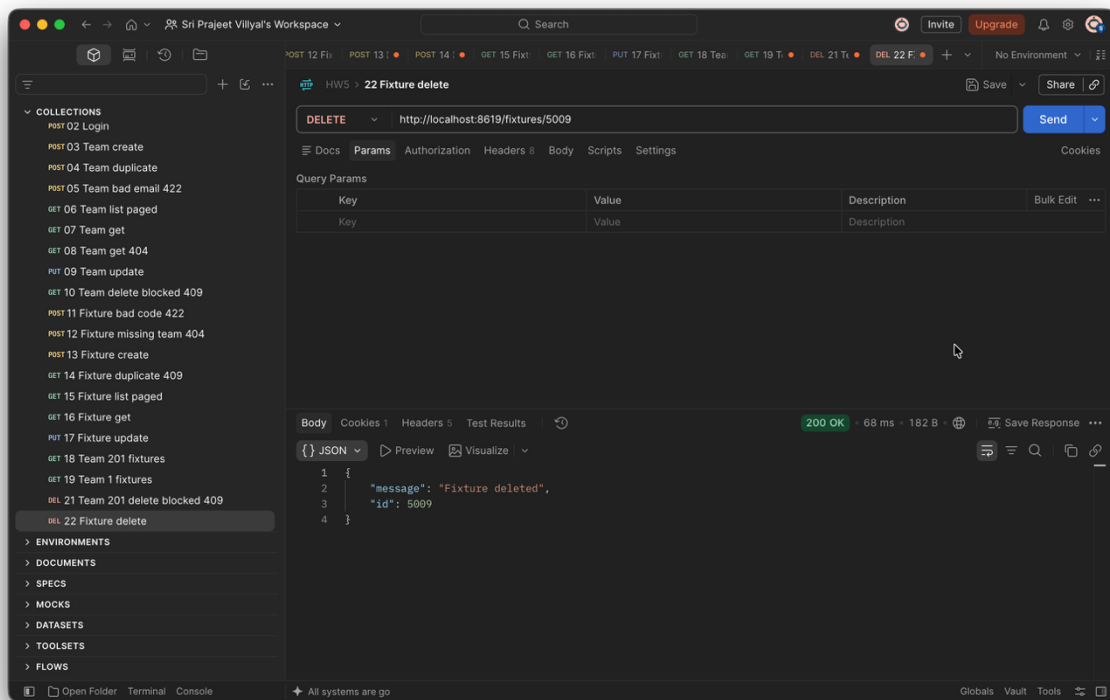
Database after the API calls



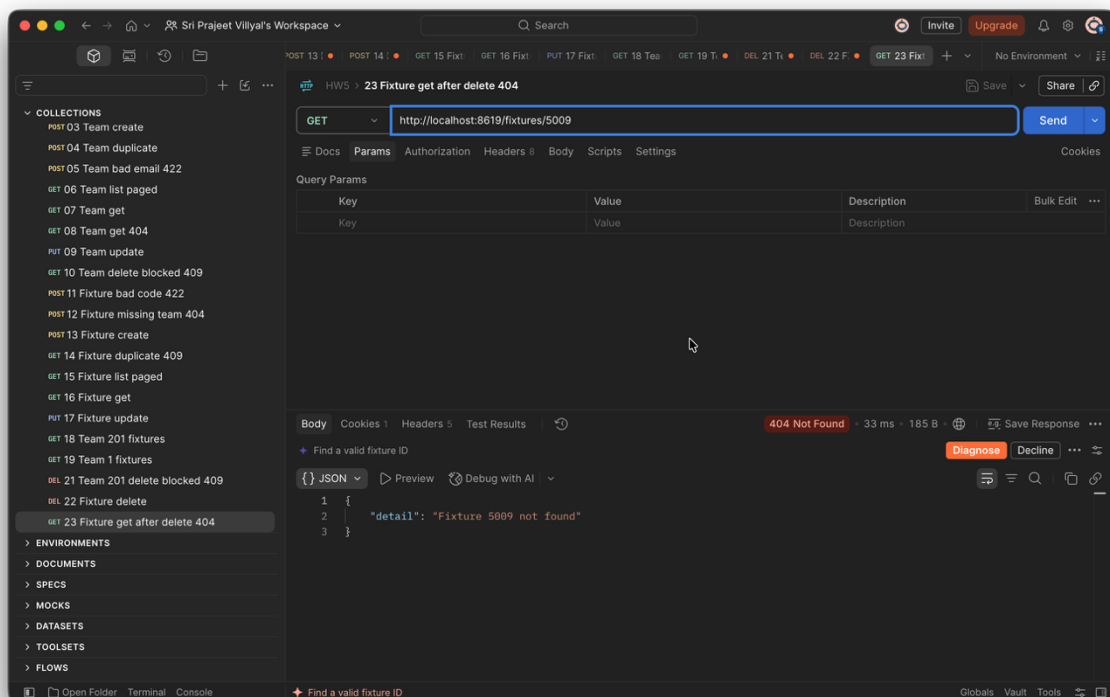
DELETE /teams/201 - 409, it still has 1 fixture



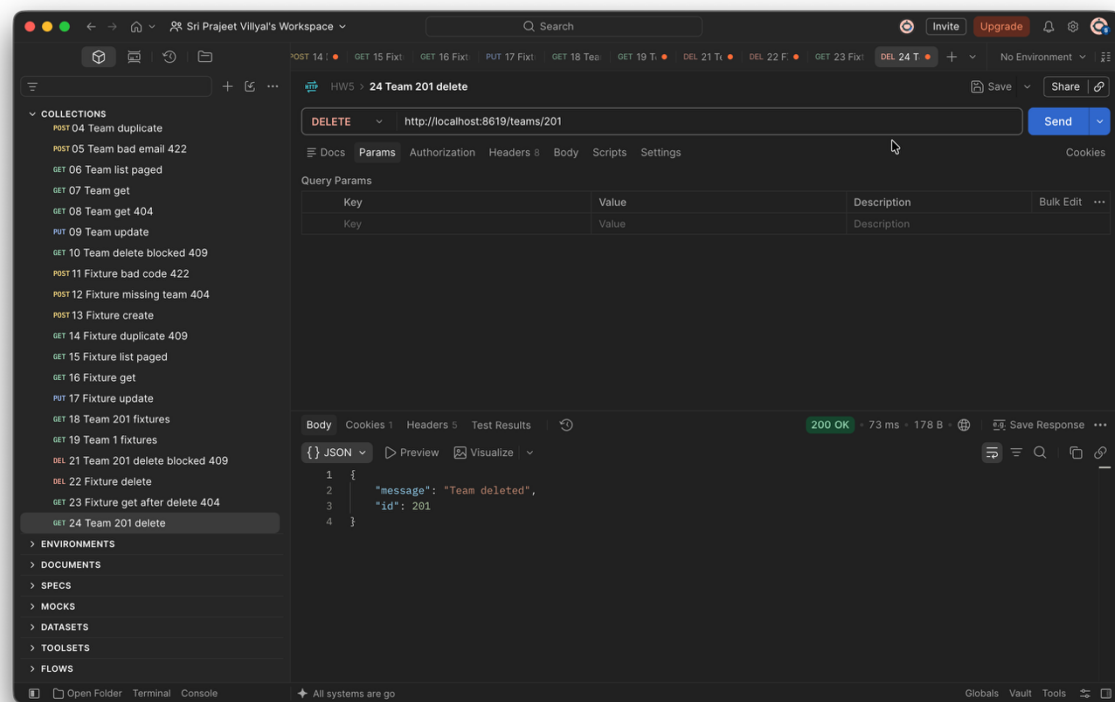
DELETE /fixtures/5009 - 200



GET /fixtures/5009 - 404 after delete



DELETE /teams/201 - 200 once it has no fixtures



3. Redux Client

The React app now keeps its fixture data in Redux Toolkit instead of local component state. The store is created in store.js with configureStore. The file fixturesSlice.js holds the fixture list and four async thunks, and each thunk calls the FastAPI backend using axios:

- fetchFixtures loads the list (GET /fixtures)
- createFixture adds a new fixture (POST /fixtures)
- updateFixture edits a fixture by its ID (PUT /fixtures/{id})
- deleteFixture removes a fixture by its ID (DELETE /fixtures/{id})

When a request succeeds, the slice adds the new fixture, replaces the edited one, or removes the deleted one in the Redux state. When a request fails, the error message from FastAPI is saved in the state and shown on the page. The Home page reads the list directly from the Redux store, so it updates right away after every change without reloading the page.

Redux setup - store.js and main.jsx

```
code > frontend > src > store > JS store.js > ...
1 // Redux store built with Redux Toolkit
2 import { configureStore } from '@reduxjs/toolkit';
3 import fixturesReducer from '../features/fixtures/fixturesSlice.js';
4 export const store = configureStore({
5   reducer: { fixtures: fixturesReducer }, // state.fixtures holds the fixture list
6 });
```

```
code > frontend > src > main.jsx
1 // Entry point: Redux Provider and router around the App
2 import React from "react";
3 import ReactDOM from "react-dom/client";
4 import { BrowserRouter } from "react-router-dom";
5 import { Provider } from "react-redux";
6 import { store } from "../store/store.js";
7 import App from "../App.jsx";
8 import "../styles.css";
9 ReactDOM.createRoot(document.getElementById("root")).render(
10   <React.StrictMode>
11     <Provider store={store}>
12       <BrowserRouter>
13         <App />
14       </BrowserRouter>
15     </Provider>
16   </React.StrictMode>
17 );
```

fixturesSlice.js - slice reducers for success and error cases

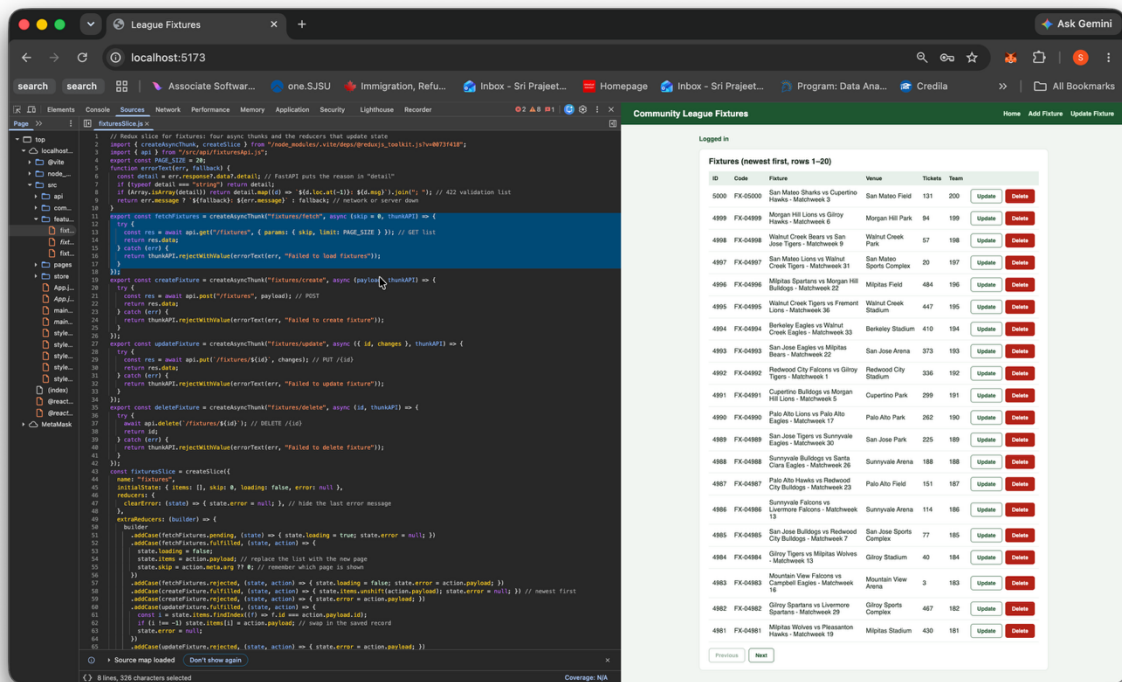
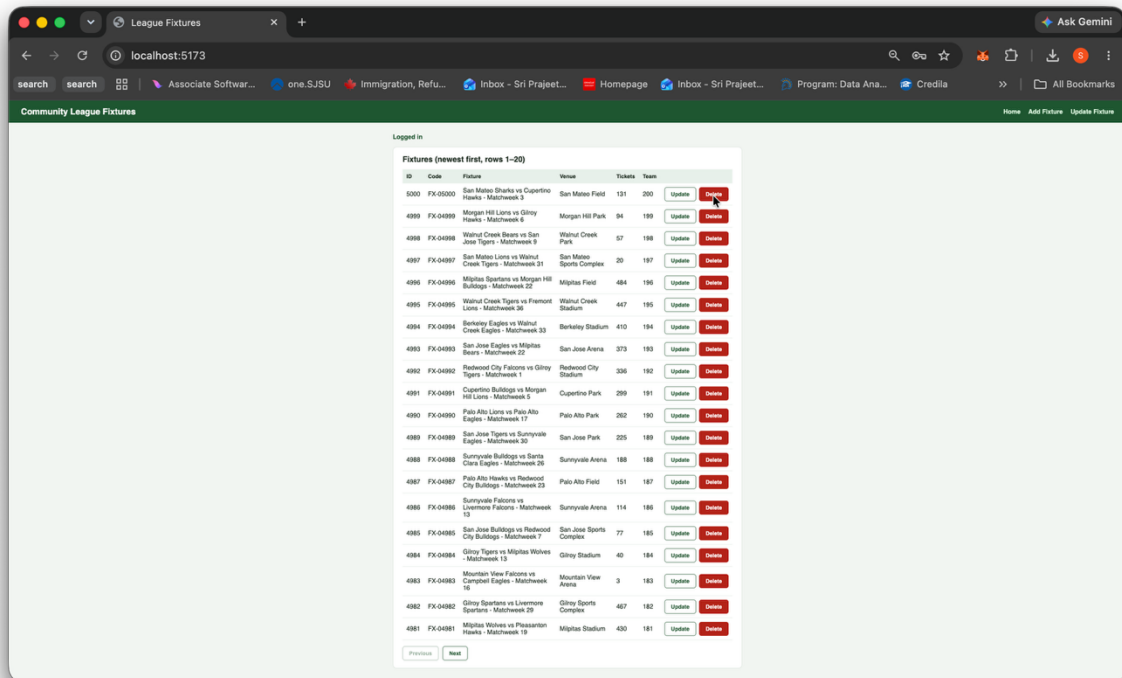
```
const fixturesSlice = createSlice({
  name: "fixtures",
  initialState: { items: [], skip: 0, loading: false, error: null },
  reducers: {
    clearError: (state) => { state.error = null; }, // hide the last error message
  },
  extraReducers: (builder) => {
    builder
      .addCase(fetchFixtures.pending, (state) => { state.loading = true; state.error = null; })
      .addCase(fetchFixtures.fulfilled, (state, action) => {
        state.loading = false;
        state.items = action.payload; // replace the list with the new page
        state.skip = action.meta.arg ?? 0; // remember which page is shown
      })
      .addCase(fetchFixtures.rejected, (state, action) => { state.loading = false; state.error = action.payload; })
      .addCase(createFixture.fulfilled, (state, action) => { state.items.unshift(action.payload); state.error = null; }) // newest first
      .addCase(createFixture.rejected, (state, action) => { state.error = action.payload; })
      .addCase(updateFixture.fulfilled, (state, action) => {
        const i = state.items.findIndex((f) => f.id === action.payload.id);
        if (i !== -1) state.items[i] = action.payload; // swap in the saved record
        state.error = null;
      })
      .addCase(updateFixture.rejected, (state, action) => { state.error = action.payload; })
      .addCase(deleteFixture.fulfilled, (state, action) => {
        state.items = state.items.filter((f) => f.id !== action.payload); // drop the deleted record
        state.error = null;
      })
      .addCase(deleteFixture.rejected, (state, action) => { state.error = action.payload; });
  },
});
export const { clearError } = fixturesSlice.actions;
export default fixturesSlice.reducer;
```

Home.jsx - reading state and the delete button

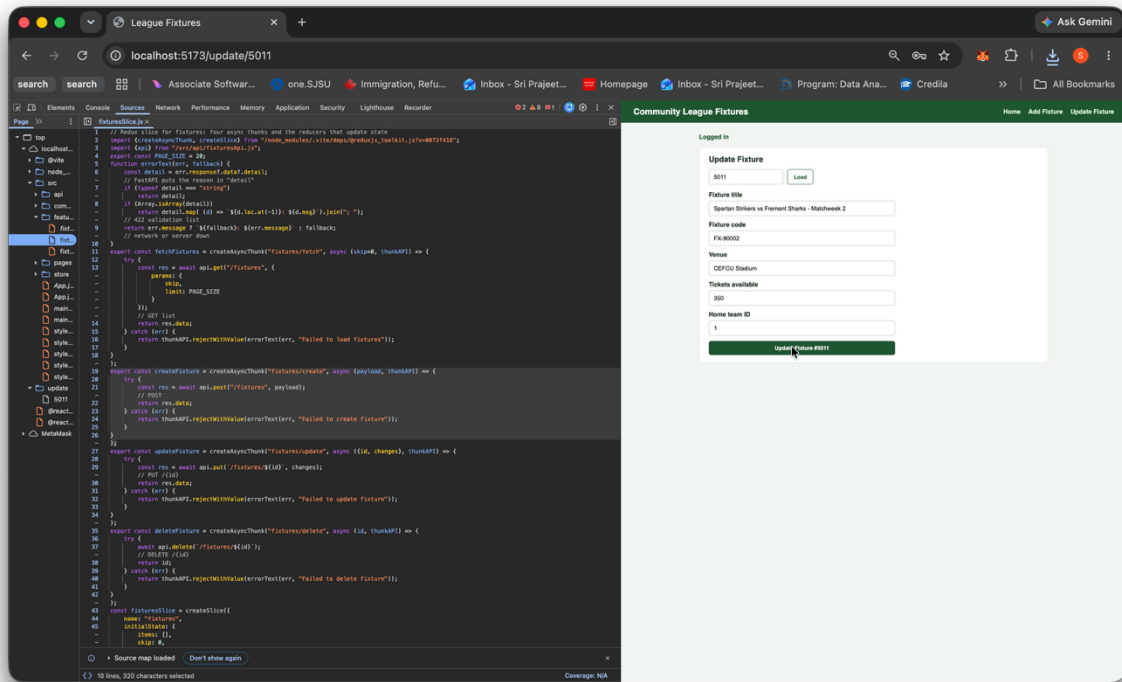
```
const { items, skip, loading, error } = useSelector((state) => state.fixtures); // data comes from the store

return (
  <div>
    <button className="btn btn-red" onClick={() => dispatch(deleteFixture(f.id))}>Delete</button>
  </div>
);
```

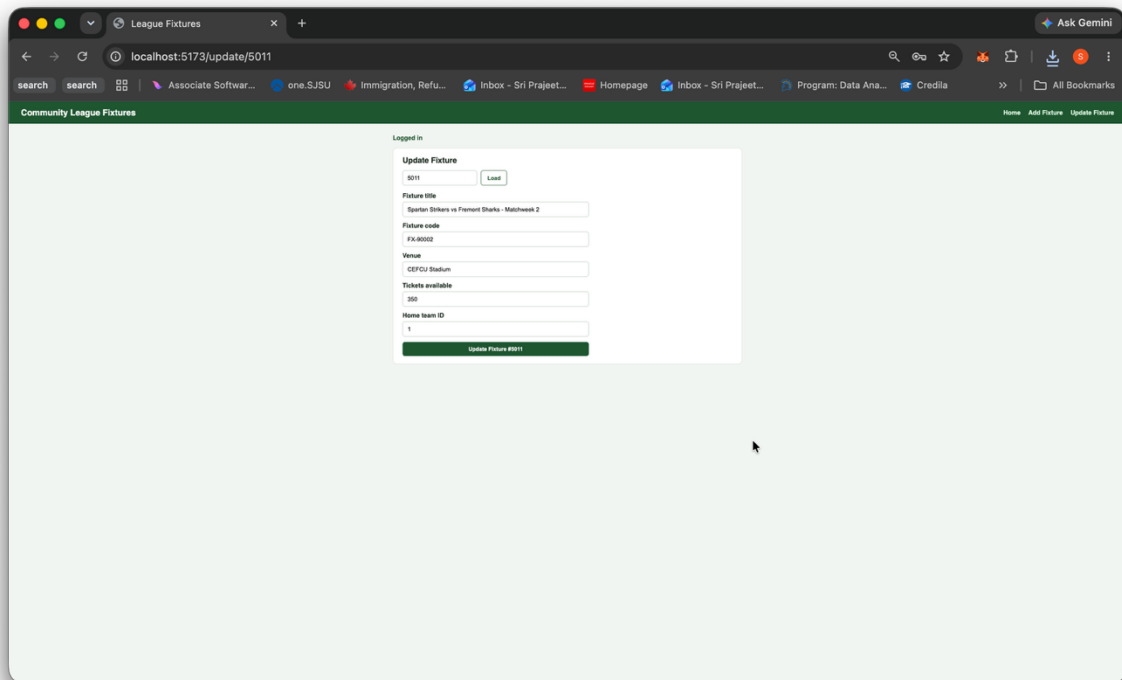
Home

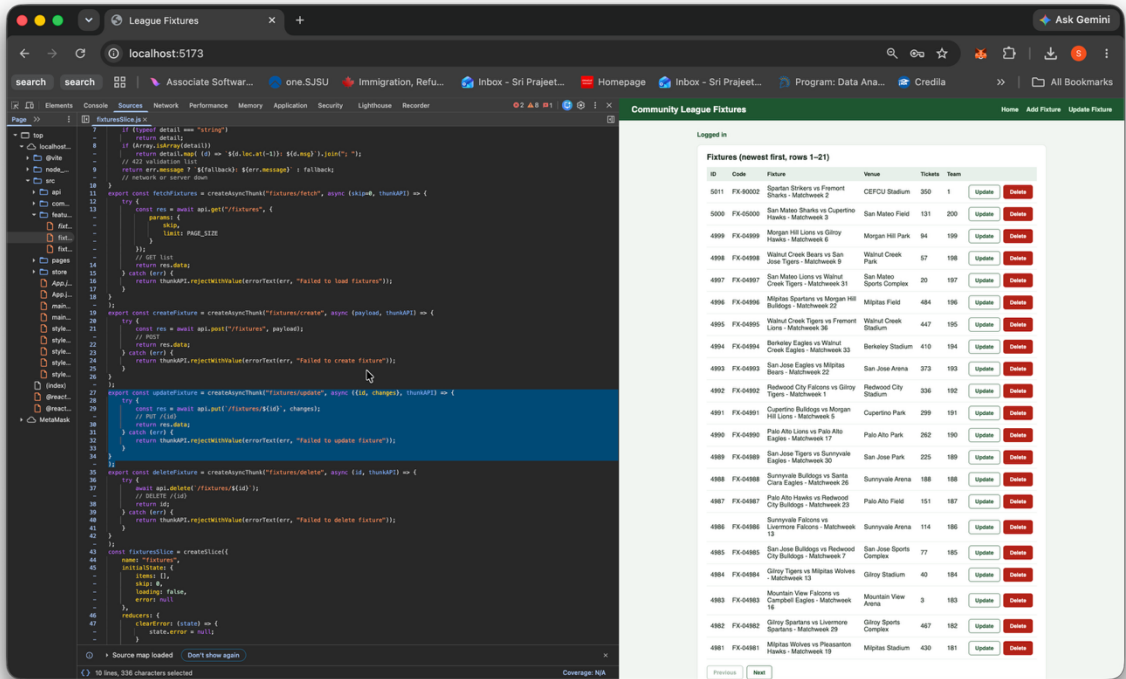


Create

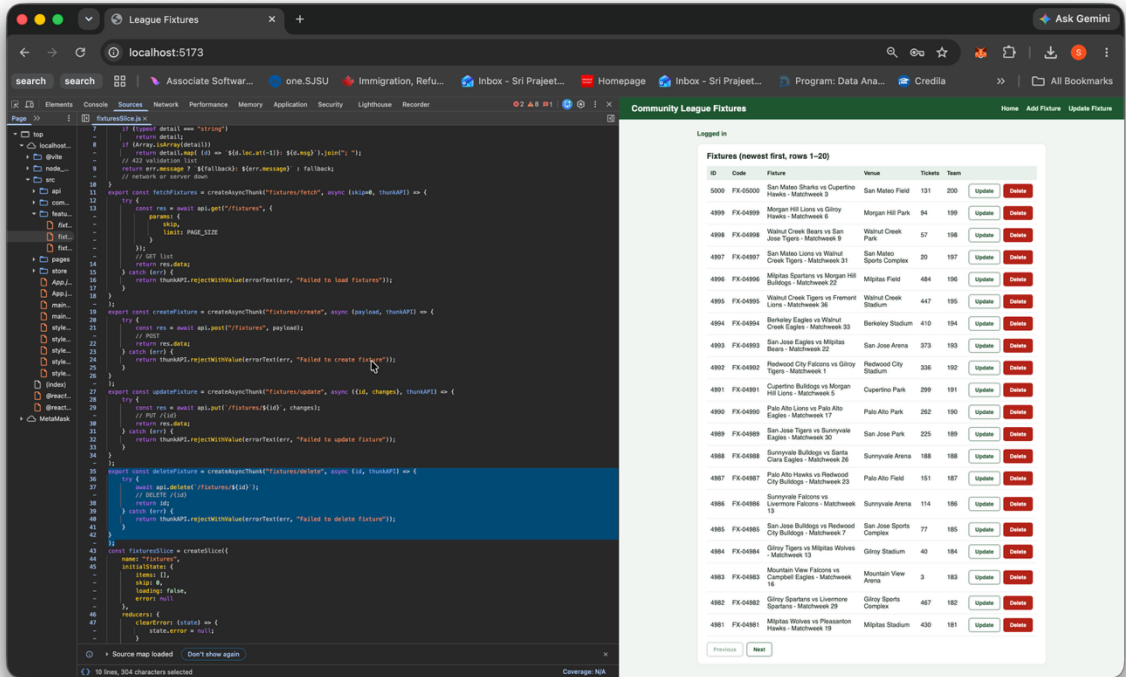


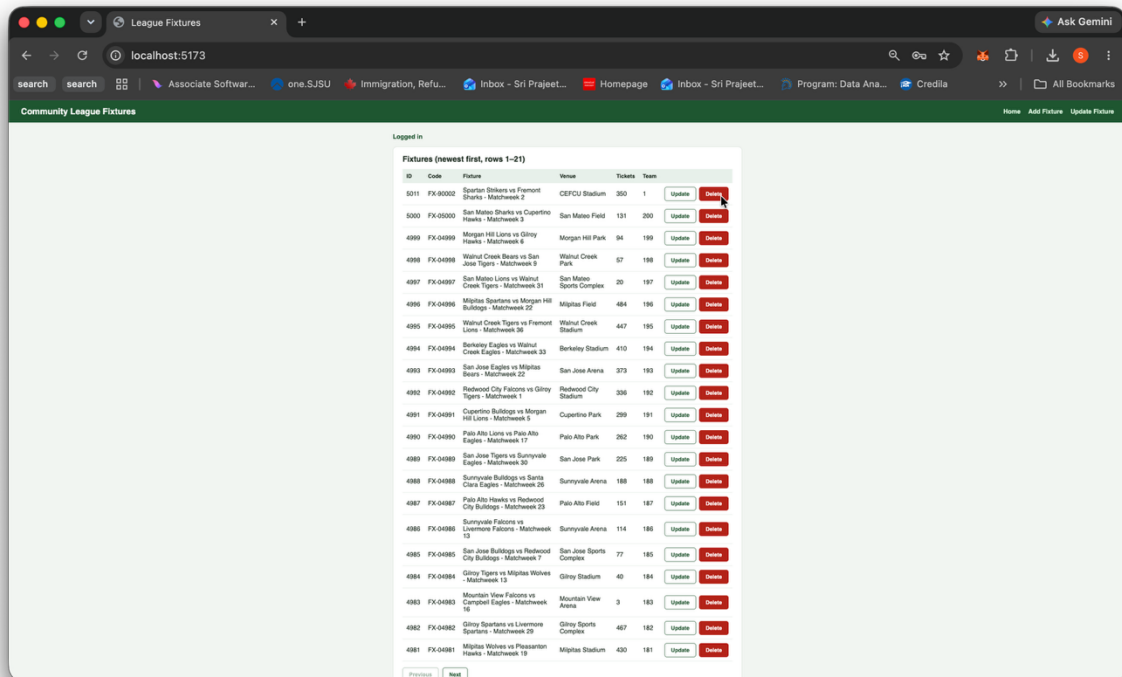
Update





Delete





Part 2 - MCP Tool Servers

In this part I built two MCP servers with FastMCP. The first one wraps a public recipe API and the second one works on my own league data in MySQL. Both servers talk to the client over STDIO and live in `code/mcp_servers/`. I tested them in MCP Inspector using `mcp dev`. Both servers send their logs to `stderr`, because `stdout` is used for the messages between the Inspector and the server.

A. TheMealDB Tutorial Server

`meals_server.py` connects to TheMealDB, a free public recipe API. It uses the free test key "1", which goes inside the web address, so there is no secret key in the code. The server has four tools. `search_meals_by_name` finds meals by name and returns up to 25 results. `meals_by_ingredient` lists meals that use a given ingredient and returns up to 50 results. `random_meal` returns one random meal with its full recipe. `meal_details` returns the full recipe for one meal ID, such as 52771. If a search finds nothing, the tool returns an empty list with a short message instead of an error. If the website cannot be reached or sends back a bad reply, the tool returns a short, clear error message.

Setup, API address with the test key, and logging to stderr

```
# HW5 Part 2A: "meals" MCP server for TheMealDB
import logging
import sys
from typing import Annotated, Any
import httpx
try:
    from mcp.server.fastmcp import FastMCP
    from mcp.server.fastmcp.exceptions import ToolError
except ImportError:
    from mcp.server.mcpserver import MCPServer as FastMCP
    from mcp.server.mcpserver.exceptions import ToolError
from pydantic import Field
# Constants
MEALDB_API_BASE = "https://www.themealdb.com/api/json/v1/1"
REQUEST_TIMEOUT = 10.0 # seconds to wait for TheMealDB before giving up
logging.basicConfig(stream=sys.stderr, level=logging.INFO, format="%(asctime)s %(levelname)s %(name)s: %(message)s")
logger = logging.getLogger("meals")
# Initialize the FastMCP server
mcp = FastMCP("meals")
```

One helper that calls TheMealDB and turns failures into clear errors

```
async def make_mealdb_request(path: str, params: dict[str, str]) -> list[dict[str, Any]] | None:
    """Call one TheMealDB endpoint and return its "meals" list (None means no matches)."""
    url = f"{MEALDB_API_BASE}/{path}"
    logger.info("GET %s %s", url, params)
    async with httpx.AsyncClient() as client:
        try:
            response = await client.get(url, params=params, timeout=REQUEST_TIMEOUT)
            response.raise_for_status()
            payload = response.json()
        except httpx.HTTPError as exc:
            logger.error("network error on %s: %s", path, exc)
            raise ToolError(f"TheMealDB request failed: {exc}") from exc
        except ValueError as exc:
            logger.error("invalid JSON from %s", path)
            raise ToolError("TheMealDB returned a response that is not valid JSON") from exc
    return payload.get("meals")
```

Building the full recipe from TheMealDB's numbered ingredient fields

```
def format_details(meal: dict[str, Any]) -> dict[str, Any]:
    """Full recipe used by meal_details and random_meal."""
    ingredients = []
    for n in range(1, 21):
        name = (meal.get(f"strIngredient{n}") or "").strip()
        if name:
            ingredients.append({"name": name, "measure": (meal.get(f"strMeasure{n}") or "").strip()})
    return {"id": meal["idMeal"], "name": meal["strMeal"], "category": meal.get("strCategory"),
            "area": meal.get("strArea"), "instructions": meal.get("strInstructions"),
            "image": meal.get("strMealThumb"), "source": meal.get("strSource"),
            "youtube": meal.get("strYoutube"), "ingredients": ingredients}
```

The four tools and the STDIO run

```
@mcp.tool()
async def search_meals_by_name(query: Annotated[str, Field(min_length=1)],
                                limit: Annotated[int, Field(ge=1, le=25)] = 5) -> dict:
    """Search TheMealDB for meals by name.

    Args:
        query: Part of a meal name, e.g. "Arrabiata"
        limit: Maximum number of meals to return (1-25)
    """
    meals = await make_mealdb_request("search.php", {"s": query.strip()})
    if not meals:
        return {"results": [], "message": f"no matches for '{query}'"} # clear, empty result
    results = [format_summary(m) for m in meals[:limit]]
    return {"count": len(results), "results": results}

@mcp.tool()
async def meals_by_ingredient(ingredient: Annotated[str, Field(min_length=1)],
                              limit: Annotated[int, Field(ge=1, le=50)] = 12) -> dict:
    """List meals that use a main ingredient.

    Args:
        ingredient: Main ingredient, e.g. "chicken"
        limit: Maximum number of meals to return (1-50)
    """
    meals = await make_mealdb_request("filter.php", {"i": ingredient.strip()})
    if not meals:
        return {"results": [], "message": f"no matches for ingredient '{ingredient}'"}
    results = [format_card(m) for m in meals[:limit]]
    return {"count": len(results), "results": results}

@mcp.tool()
async def random_meal() -> dict:
    """Return one random meal with its full recipe."""
    meals = await make_mealdb_request("random.php", {})
    if not meals:
        return {"message": "no matches"}
    return format_details(meals[0])

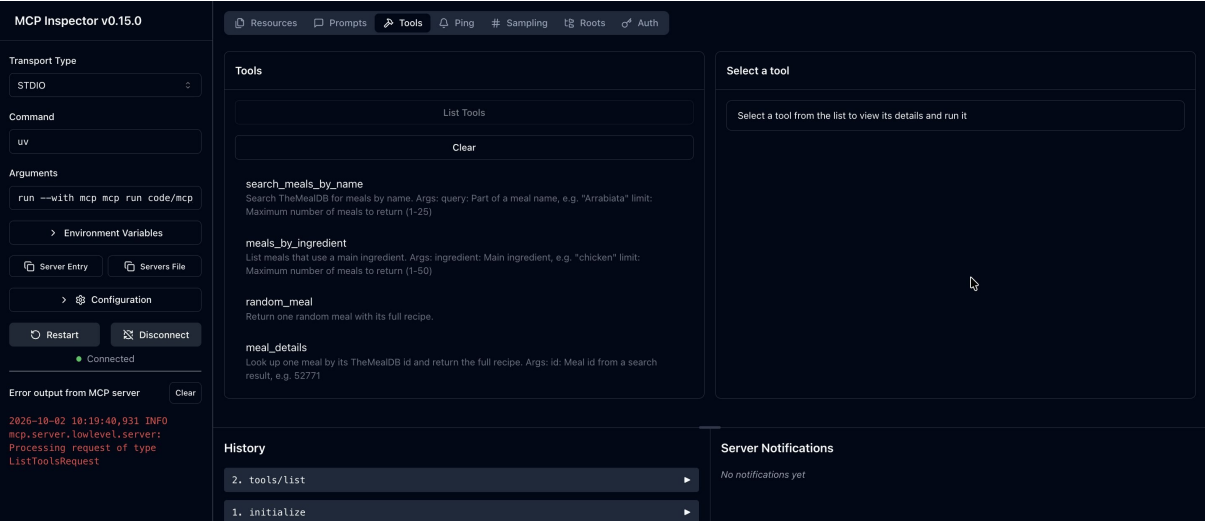
@mcp.tool()
async def meal_details(id: str | int) -> dict:
    """Look up one meal by its TheMealDB id and return the full recipe.

    Args:
        id: Meal id from a search result, e.g. 52771
    """
    meal_id = str(id).strip()
    if not meal_id.isdigit():
        raise ToolError(f"id must be a numeric meal id, got '{id}'") # shown as a clean error in Inspector
    meals = await make_mealdb_request("lookup.php", {"i": meal_id})
    if not meals:
        return {"message": f"no meal with id {meal_id}"}
    return format_details(meals[0])

def main() -> None:
    mcp.run(transport="stdio")

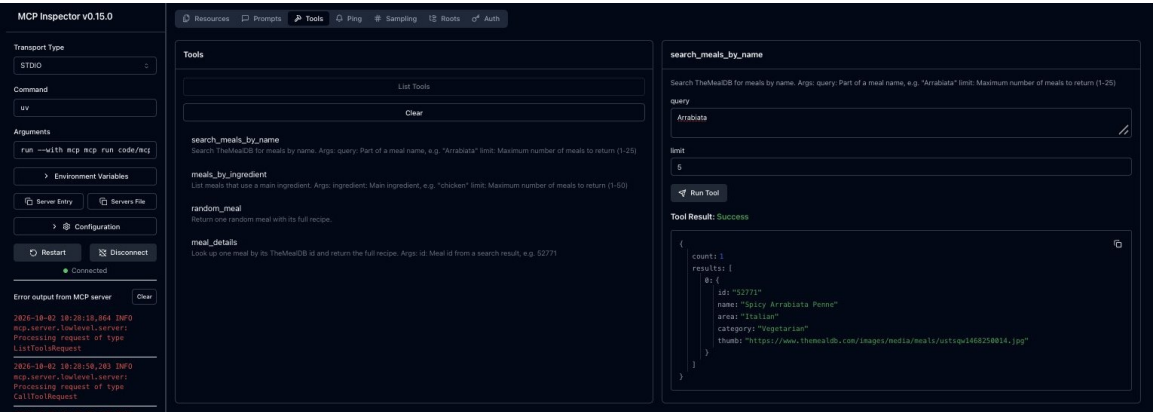
if __name__ == "__main__":
    main()
```

Inspector connected over STDIO with the four tools listed

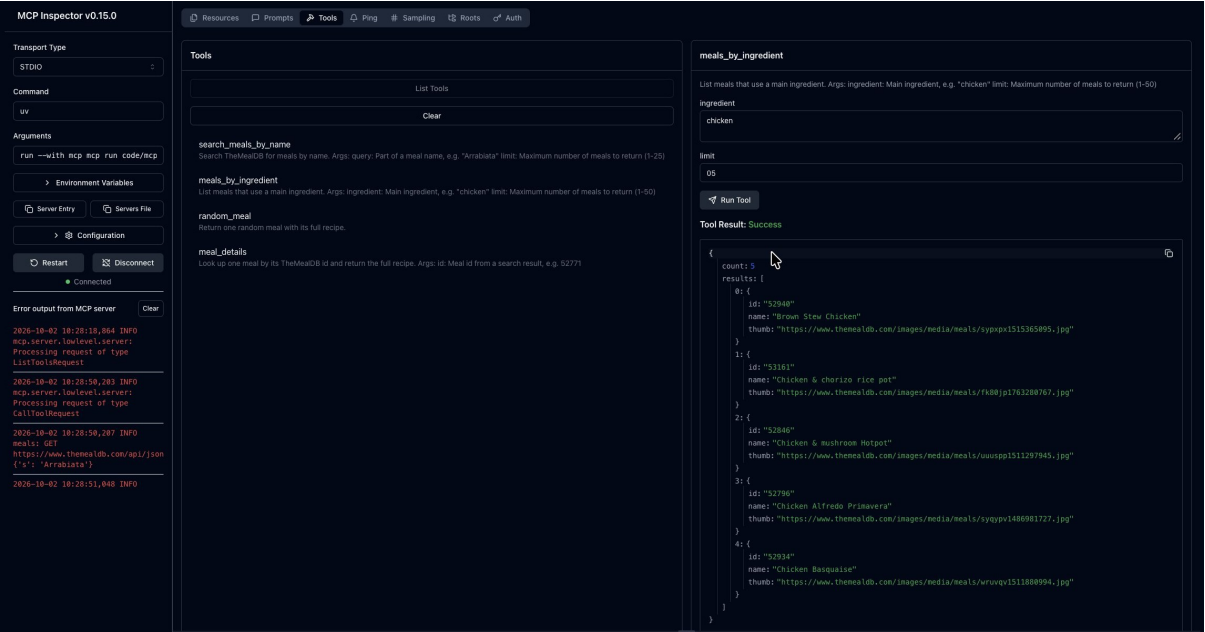


search_meals_by_name with "Arrabiata" and limit 5

It found one meal, Spicy Arrabiata Penne. The log on the left shows the request the server sent to TheMealDB.

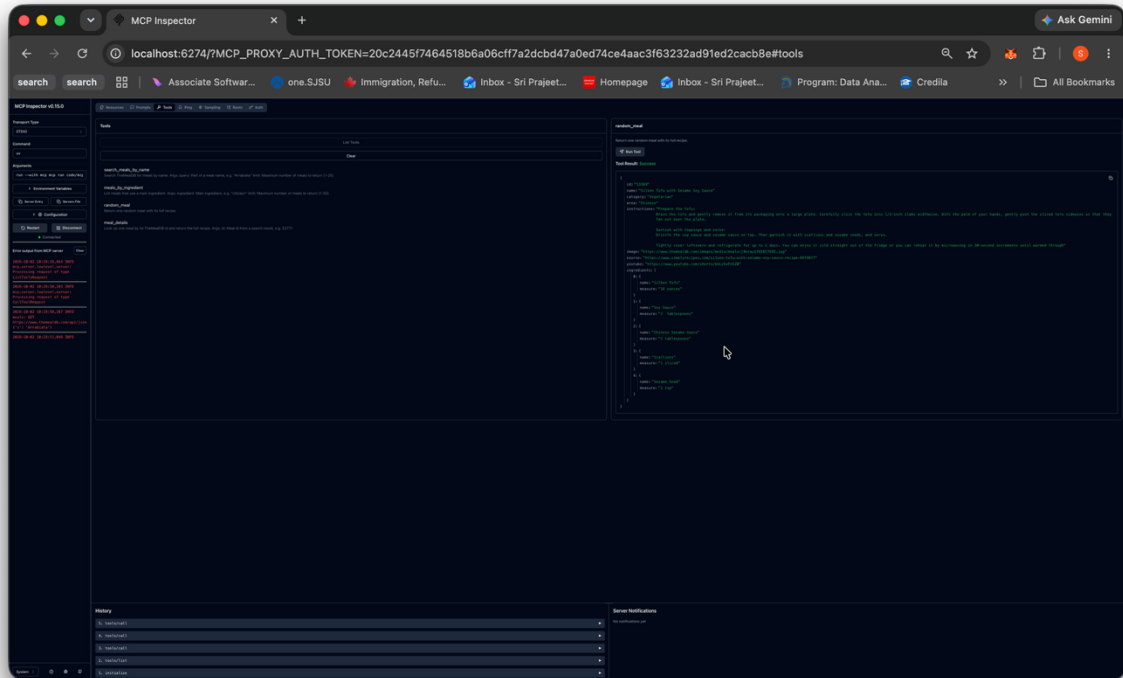


meals_by_ingredient with "chicken" and limit 5



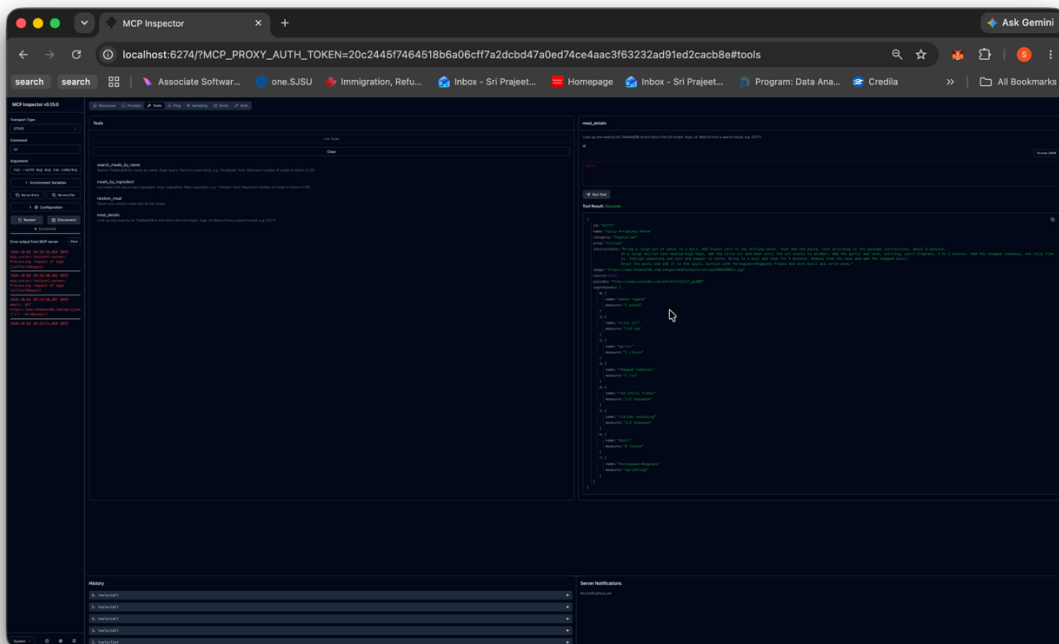
random_meal

Each run gives a different meal. This run returned Silken Tofu with Sesame Soy Sauce with its 5 ingredients.



meal_details with ID 52771

The full recipe for Spicy Arrabiata Penne, with the instructions, image, YouTube link and all 8 ingredients and their measures.



The raw outputs of these four calls are saved in reports/hw05/raw/ as p2a_search_meals_by_name.json, p2a_meals_by_ingredient.json, p2a_random_meal.json and p2a_meal_details.json.

B. Domain MCP Server

league_server.py works on my own fixtures and teams tables and has exactly three tools. search_fixtures is the search tool: it finds fixtures whose title or venue contains some text, for example "Hawks". fixture_details is the detail lookup: it takes one fixture code such as FX-00001 and returns that fixture together with its home team. tickets_by_city is the aggregate: it groups fixtures by the home team's city and returns how many fixtures each city has, the total tickets available and the average, with the highest total first.

The server reads the database connection from the same code/api/.env file as my FastAPI app, so the password is not written in the code. The queries use placeholders, so whatever the user types is never pasted into the SQL, and the database connection gives up after 5 seconds instead of hanging.

One response envelope for all three tools

All three tools reply in the same shape, {ok, data, error}. When a call works, ok is true, data holds the result and error is null. When it fails, ok is false, data is null and error has a code and a message. I defined this once in envelope.py so I can reuse the same two functions for execute_tool in Part 4. I used three error codes: INVALID_INPUT when the arguments are wrong (checked before the database is touched), NOT_FOUND when a fixture code is valid but does not exist, and DB_ERROR when the database cannot be reached.

```
code > mcp_servers > envelope.py > ...
1 # Shared {ok, data, error} response envelope for the domain MCP tools (reused by execute_tool in Part 4)
2 from typing import Any
3 def ok(data: Any) -> dict:
4     """Success: data holds the result and error is null."""
5     return {"ok": True, "data": data, "error": None}
6 def fail(code: str, message: str) -> dict:
7     """Failure: data is null and error says what went wrong."""
8     return {"ok": False, "data": None, "error": {"code": code, "message": message}}
```

Setup, database settings from .env, and logging to stderr

```
code > mcp_servers > league_server.py > read_database_url
1 # HW5 Part 2B: "league" MCP server over s3319_rel with 3 tools (search, detail, aggregate), run over STDIO
2 import logging
3 import os
4 import re
5 import sys
6 from pathlib import Path
7 from urllib.parse import unquote, urlparse
8 try:
9     from mcp.server.fastmcp import FastMCP
10 except ImportError:
11     from mcp.server.mcpserver import MCPServer as FastMCP
12 from envelope import fail, ok
13 # Constants
14 ENV_FILE = Path(__file__).resolve().parents[1] / "api" / ".env" # same DATABASE_URL as the FastAPI app
15 CODE_PATTERN = re.compile(r"^FX-\d{5}$") # fixture codes look like FX-00001
16 DB_TIMEOUT = 5 # seconds to wait for MySQL before giving up
17 logging.basicConfig(stream=sys.stderr, level=logging.INFO, format="%(asctime)s %(levelname)s %(name)s: %(message)s")
18 logger = logging.getLogger("league")
19 # Initialize the FastMCP server
20 mcp = FastMCP("league")
21 def read_database_url() -> str:
22     """Take DATABASE_URL from the environment, else from code/api/.env."""
23     url = os.getenv("DATABASE_URL")
24     if not url and ENV_FILE.exists():
25         for line in ENV_FILE.read_text().splitlines():
26             if line.startswith("DATABASE_URL="):
27                 url = line.split("=", 1)[1].strip()
28     if not url:
29         raise RuntimeError(f"DATABASE_URL not set and not found in {ENV_FILE}")
30     return url
```

Read only queries for the three tools

```
class MySQLStore:
    """Read-only access to fixtures and teams in s3319_rel."""

    def __init__(self, url: str):
        parts = urlparse(url)
        self.config = {"host": parts.hostname or "localhost", "port": parts.port or 3306,
                       "user": unquote(parts.username or ""), "password": unquote(parts.password or ""),
                       "database": parts.path.lstrip("/")}

    def query(self, sql: str, params: tuple) -> list[dict]:
        import pymysql # imported here so the server still starts and lists tools without it
        conn = pymysql.connect(**self.config, connect_timeout=DB_TIMEOUT, read_timeout=DB_TIMEOUT,
                               cursorclass=pymysql.cursors.DictCursor)
        try:
            with conn.cursor() as cur:
                cur.execute(sql, params)
                return list(cur.fetchall())
        finally:
            conn.close()

    def search_fixtures(self, text: str, limit: int) -> list[dict]:
        like = f"%{text}%"
        return self.query(
            "SELECT f.fixture_code, f.fixture_title, f.venue, f.tickets_available, t.name AS home_team "
            "FROM fixtures f JOIN teams t ON t.id = f.home_team_id "
            "WHERE f.fixture_title LIKE %s OR f.venue LIKE %s ORDER BY f.id LIMIT %s", (like, like, limit))

    def get_fixture(self, code: str) -> dict | None:
        rows = self.query(
            "SELECT f.id, f.fixture_code, f.fixture_title, f.venue, f.tickets_available, f.created_at, "
            "t.id AS team_id, t.name AS team_name, t.city AS team_city, t.contact_email AS team_email "
            "FROM fixtures f JOIN teams t ON t.id = f.home_team_id WHERE f.fixture_code = %s", (code,))
        return rows[0] if rows else None

    def tickets_by_city(self, top_n: int) -> list[dict]:
        return self.query(
            "SELECT t.city, COUNT(*) AS fixtures, SUM(f.tickets_available) AS total_tickets, "
            "ROUND(AVG(f.tickets_available), 1) AS avg_tickets "
            "FROM fixtures f JOIN teams t ON t.id = f.home_team_id "
            "GROUP BY t.city ORDER BY total_tickets DESC, t.city LIMIT %s", (top_n,))
```

Wrapping every database call in the envelope

```
_store = None
def get_store():
    """Create the store on first use so a missing DB shows up as an envelope error, not a crash."""
    global _store
    if _store is None:
        _store = MySQLStore(read_database_url())
    return _store
def run_query(action) -> dict:
    """Run one store call and wrap any database problem in the envelope."""
    try:
        return ok(action(get_store()))
    except ModuleNotFoundError:
        logger.error("pymysql is not installed")
        return fail("DB_ERROR", "pymysql is not installed; start with: mcp dev league_server.py --with pymysql")
    except Exception as exc:
        logger.error("database error: %s", exc)
        return fail("DB_ERROR", f"database request failed: {exc}")
```

The three tools with their input checks

```
@mcp.tool()
async def search_fixtures(query: str, limit: int = 10) -> dict:
    """Search fixtures whose title or venue contains the text.

    Args:
        query: Text to look for, e.g. "Hawks" or "Field" (at least 2 characters)
        limit: Maximum number of fixtures to return (1-50)
    """
    text = query.strip()
    if len(text) < 2:
        return fail("INVALID_INPUT", "query must be at least 2 characters")
    if not 1 <= limit <= 50:
        return fail("INVALID_INPUT", f"limit must be between 1 and 50, got {limit}")
    logger.info("search_fixtures query=%r limit=%d", text, limit)
    result = run_query(lambda s: [to_json(r) for r in s.search_fixtures(text, limit)])
    if result["ok"]:
        result["data"] = {"count": len(result["data"]), "results": result["data"]}
    return result

@mcp.tool()
async def fixture_details(fixture_code: str) -> dict:
    """Look up one fixture by its code, with its home team.

    Args:
        fixture_code: Fixture code in the form FX-00001
    """
    code = fixture_code.strip().upper()
    if not CODE_PATTERN.match(code):
        return fail("INVALID_INPUT", f"fixture_code must look like FX-00001, got '{fixture_code}'")
    logger.info("fixture_details code=%s", code)
    result = run_query(lambda s: s.get_fixture(code))
    if result["ok"] and result["data"] is None:
        return fail("NOT_FOUND", f"no fixture with code {code}")
    if result["ok"]:
        result["data"] = to_json(result["data"])
    return result

@mcp.tool()
async def tickets_by_city(top_n: int = 5) -> dict:
    """Aggregate: fixtures and tickets available per home-team city, highest total first.

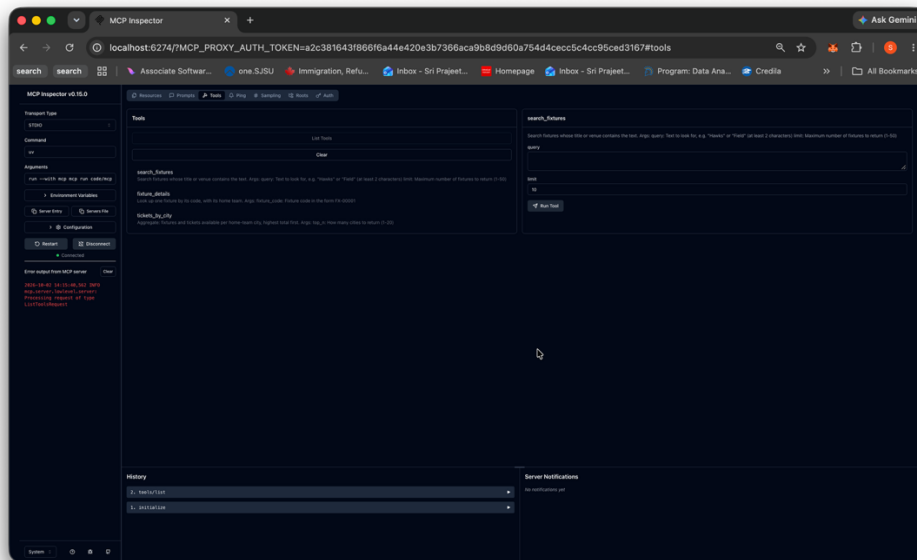
    Args:
        top_n: How many cities to return (1-20)
    """
    if not 1 <= top_n <= 20:
        return fail("INVALID_INPUT", f"top_n must be between 1 and 20, got {top_n}")
    logger.info("tickets_by_city top_n=%d", top_n)
    return run_query(lambda s: [to_json(r) for r in s.tickets_by_city(top_n)])

def main() -> None:
    mcp.run(transport="stdio")
if __name__ == "__main__":
    main()
```

Testing in MCP Inspector

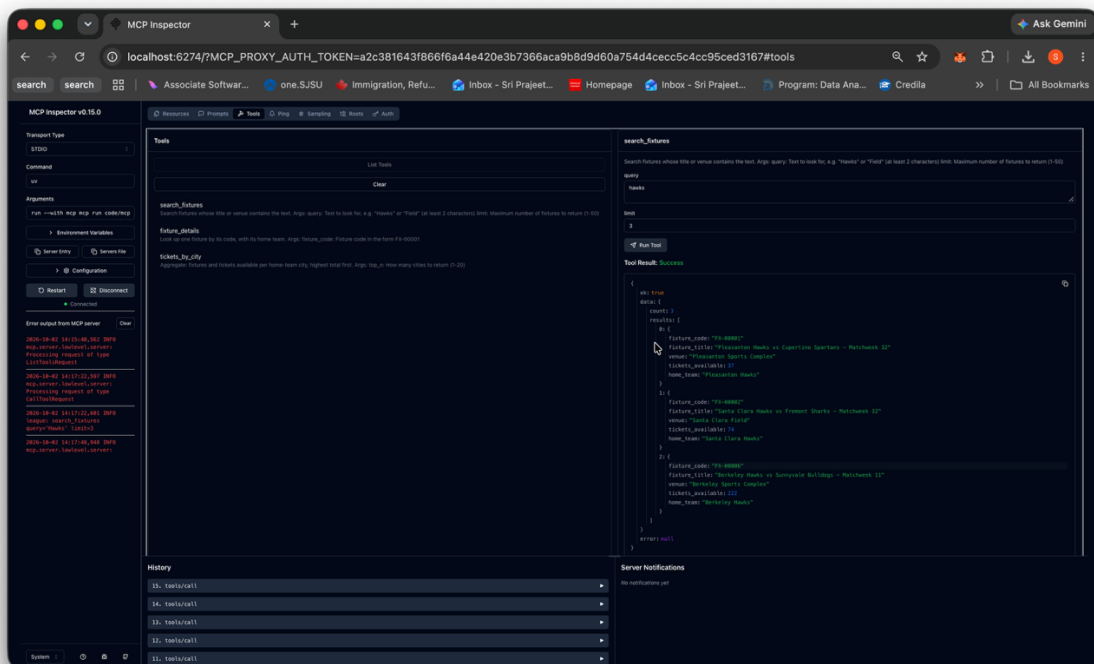
For each tool I ran one call that should work and one call that I made invalid on purpose. The invalid calls still show "Tool Result: Success" in the Inspector because the tool itself ran fine and sent back a clean envelope with ok false, instead of crashing. These three rejected calls are the ones I use again in Part 3.

Inspector connected over STDIO with the three domain tools listed



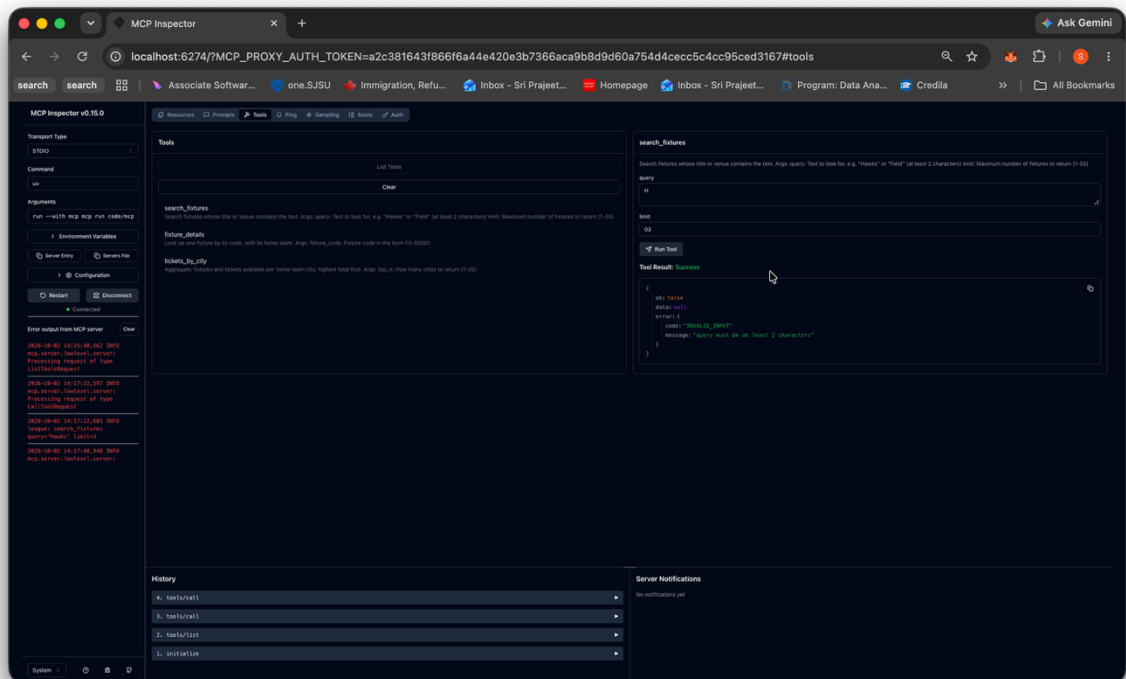
search_fixtures, valid: query "Hawks", limit 3

ok is true, it returned 3 fixtures and error is null. The three tools of the server are listed on the left.



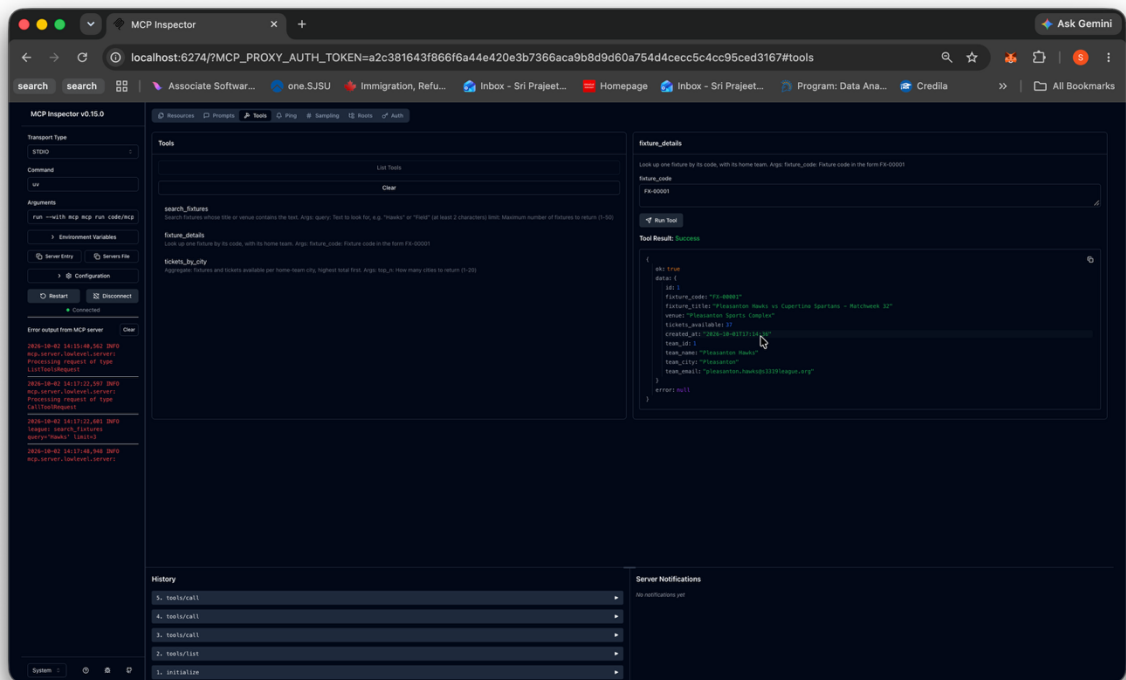
search_fixtures, invalid: query "H"

Rejected with INVALID_INPUT because the search text must be at least 2 characters.



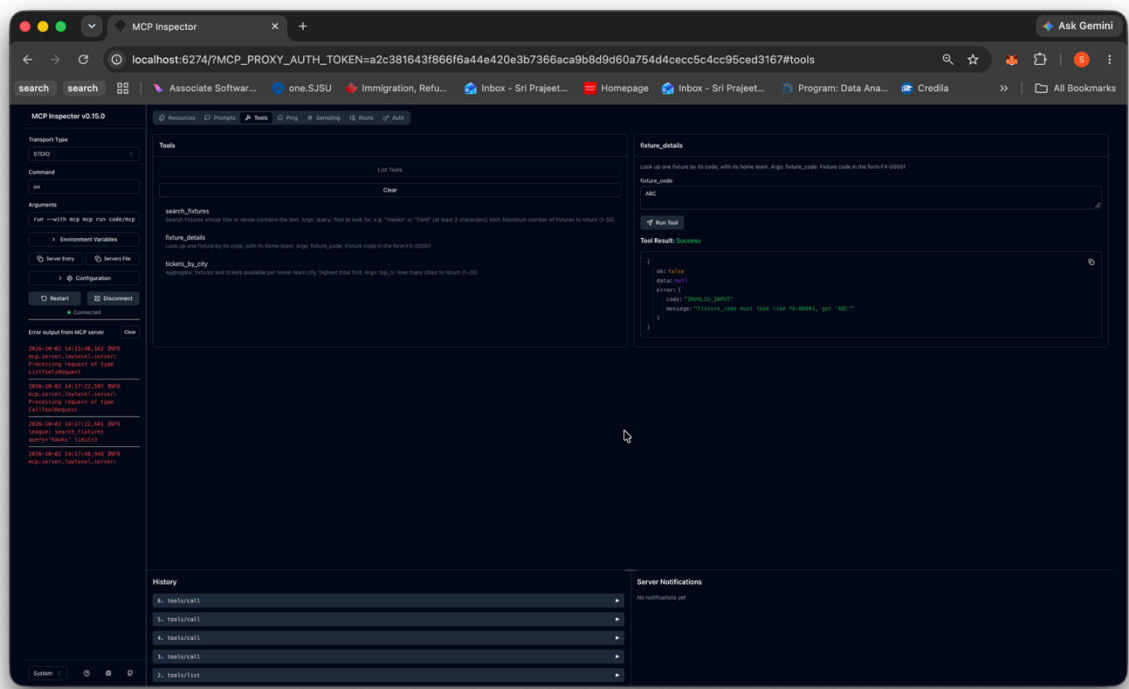
fixture_details, valid: FX-00001

Returns Pleasanton Hawks vs Cupertino Spartans with 37 tickets, and its home team Pleasanton Hawks with the team email.



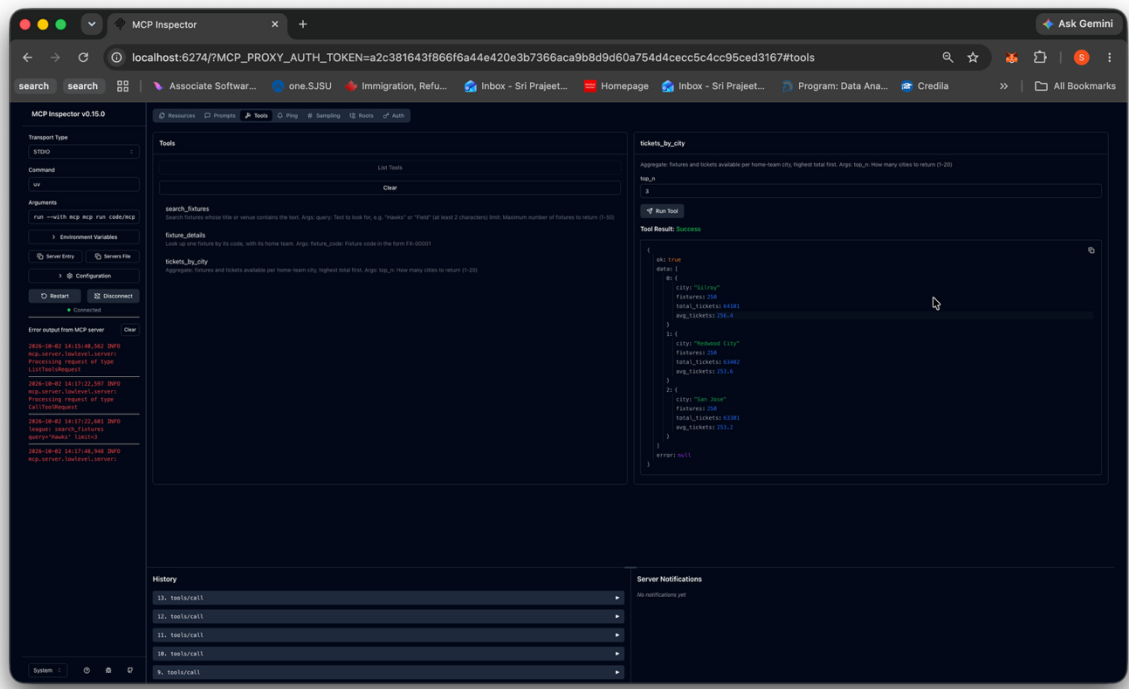
fixture_details, invalid: "ABC"

Rejected with INVALID_INPUT because a fixture code must look like FX-00001.



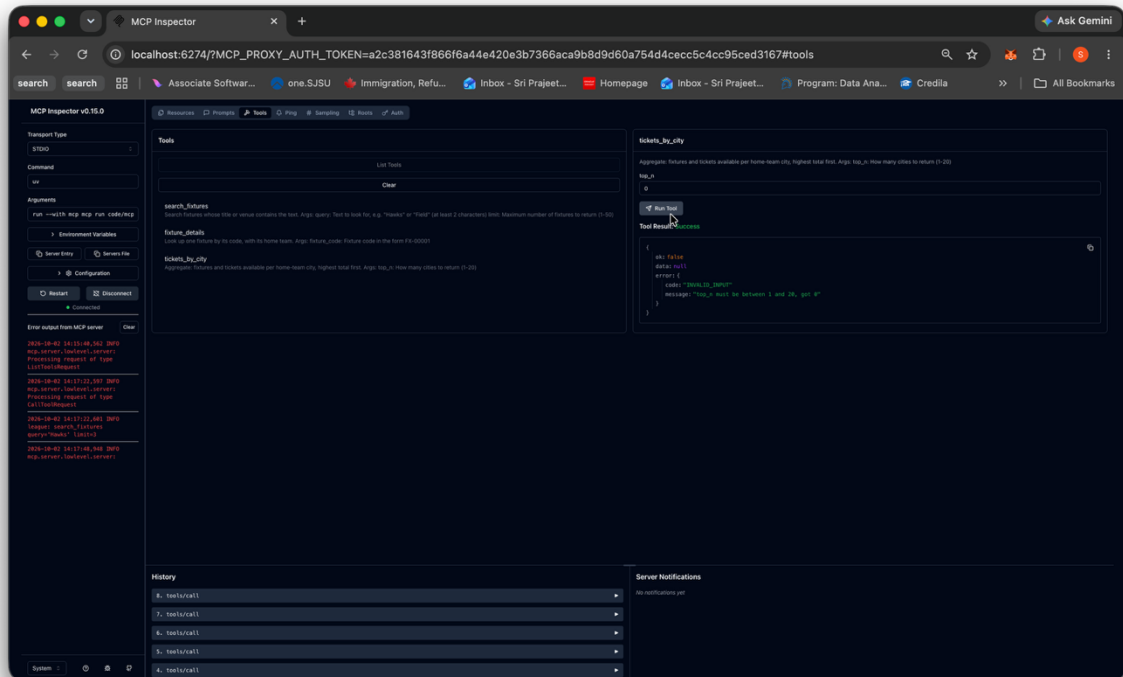
tickets_by_city, valid: top_n 3

The top three cities by total tickets are Gilroy (64,101), Redwood City (63,402) and San Jose (63,301), each with 250 fixtures.



tickets_by_city, invalid: top_n 0

Rejected with INVALID_INPUT because top_n has to be between 1 and 20.



The raw outputs of all six calls are saved in reports/hw05/raw/ as p2b_search_valid.json, p2b_search_invalid.json, p2b_details_valid.json, p2b_details_invalid.json, p2b_aggregate_valid.json and p2b_aggregate_invalid.json.

Part 3 - Tool Contracts Under Stress

This part tests how my three domain tools handle bad input and database failures. I documented the rejected calls, added timeouts and retries, ran 150 calls with simulated failures, and reviewed whether the retry policy suits an interactive assistant.

18. Rejected calls from Part 2B

These are the three invalid calls from Part 2B. Each tool checks its input in league_server.py before touching the database, so bad input never reaches MySQL. The tool returns ok false and an INVALID_INPUT error explaining what was wrong. For each call I show the check in my code, the expected input, the rejected input and the returned error. The errors are saved in reports/hw05/raw/p2b_search_invalid.json, p2b_details_invalid.json and p2b_aggregate_invalid.json.

search_fixtures

```
115 @mcp.tool()
116 async def search_fixtures(query: str, limit: int = 10) -> dict:
117     """Search fixtures whose title or venue contains the text.
118
119     Args:
120         query: Text to look for, e.g. "Hawks" or "Field" (at least 2 characters)
121         limit: Maximum number of fixtures to return (1-50)
122     """
123     text = query.strip()
124     if len(text) < 2:
125         return fail("INVALID_INPUT", "query must be at least 2 characters")
126     if not 1 <= limit <= 50:
127         return fail("INVALID_INPUT", f"limit must be between 1 and 50, got {limit}")
128     logger.info("search_fixtures query=%r limit=%d", text, limit)
129     result = run_query(lambda s: [to_json(r) for r in s.search_fixtures(text, limit)])
130     if result["ok"]:
131         result["data"] = {"count": len(result["data"]), "results": result["data"]}
132     return result
```

Expected schema: {"query": {"type": "string", "minLength": 2}, "limit": {"type": "integer", "minimum": 1, "maximum": 50, "default": 10}}, required: ["query"]

Rejected input: {"query": "H", "limit": 3}

Returned error: {"ok": false, "data": null, "error": {"code": "INVALID_INPUT", "message": "query must be at least 2 characters"}}

Why it was rejected: "H" is only 1 character, so the check on line 123 failed. The search needs at least 2 characters.

fixture_details

```
CODE_PATTERN = re.compile(r"^[FX-]\d{5}$") # fixture codes look like FX-00001
```

```
133 @mcp.tool()
134 async def fixture_details(fixture_code: str) -> dict:
135     """Look up one fixture by its code, with its home team.
136
137     Args:
138         fixture_code: Fixture code in the form FX-00001
139     """
140     code = fixture_code.strip().upper()
141     if not CODE_PATTERN.match(code):
142         return fail("INVALID_INPUT", f"fixture_code must look like FX-00001, got '{fixture_code}'")
143     logger.info("fixture_details code=%s", code)
144     result = run_query(lambda s: s.get_fixture(code))
145     if result["ok"] and result["data"] is None:
146         return fail("NOT_FOUND", f"no fixture with code {code}")
147     if result["ok"]:
148         result["data"] = to_json(result["data"])
149     return result
```

Expected schema: {"fixture_code": {"type": "string", "pattern": "^[FX-]\d{5}\$"}}, required: ["fixture_code"]

Rejected input: {"fixture_code": "ABC"}

Returned error: {"ok": false, "data": null, "error": {"code": "INVALID_INPUT", "message": "fixture_code must look like FX-00001, got 'ABC'"}}

Why it was rejected: "ABC" does not match the FX-00001 pattern on line 17, so the check on line 140 failed.

tickets_by_city

```
150 @mcp.tool()
151 async def tickets_by_city(top_n: int = 5) -> dict:
152     """Aggregate: fixtures and tickets available per home-team city, highest total first.
153
154     Args:
155         top_n: How many cities to return (1-20)
156     """
157     if not 1 <= top_n <= 20:
158         return fail("INVALID_INPUT", f"top_n must be between 1 and 20, got {top_n}")
159     logger.info("tickets_by_city top_n=%d", top_n)
160     return run_query(lambda s: [to_json(r) for r in s.tickets_by_city(top_n)])
161 def main() -> None:
162     mcp.run(transport="stdio")
163 if __name__ == "__main__":
164     main()
```

Expected schema: {"top_n": {"type": "integer", "minimum": 1, "maximum": 20, "default": 5}}

Rejected input: {"top_n": 0}

Returned error: {"ok": false, "data": null, "error": {"code": "INVALID_INPUT", "message": "top_n must be between 1 and 20, got 0"}}

Why it was rejected: 0 is outside the 1 to 20 range checked on line 156.

19. Timeouts and retries

Each database call waits at most 5 seconds before giving up. If a call fails because of a short lived problem, like a dropped connection, it tries again, up to 3 times in total. It waits 100 ms before the second try and 200 ms before the third. Errors that won't fix themselves, like a wrong password, aren't retried. If all 3 tries fail, the tool returns a clean UNAVAILABLE error instead of crashing.

retry.py - bounded exponential backoff retry

```
code > mcp_servers > ⚙️ retry.py > 🔍 retry_call
1 # Timeouts are set on each database call; this file adds a bounded exponential-backoff retry around it
2 import logging
3 import time
4 from dataclasses import dataclass
5 logger = logging.getLogger("league.retry")
6
7 class TransientError(Exception):
8     """A failure that may go away if we try again (lost connection, timeout, injected fault)."""
9
10 class RetryExhausted(Exception):
11     """Every allowed attempt failed."""
12     def __init__(self, attempts: int, last_error: Exception):
13         super().__init__(f"failed after {attempts} attempts: {last_error}")
14         self.attempts = attempts
15         self.last_error = last_error
16
17 @dataclass(frozen=True)
18 class RetryPolicy:
19     max_attempts: int = 3 # first try plus 2 retries
20     base_delay: float = 0.1 # seconds to wait before the first retry
21     max_delay: float = 1.0 # the wait never grows past this
22     def delay(self, attempt: int) -> float:
23         """Wait after a failed attempt: 0.1s, 0.2s, 0.4s ... capped at max_delay."""
24         return min(self.max_delay, self.base_delay * 2 ** (attempt - 1))
25
26 def retry_call(fn, policy: RetryPolicy, is_retryable, sleep=time.sleep):
27     """Run fn; retry only retryable errors; return (result, attempts) or raise RetryExhausted."""
28     for attempt in range(1, policy.max_attempts + 1):
29         try:
30             return fn(), attempt
31         except Exception as exc:
32             if not is_retryable(exc):
33                 raise
34             if attempt == policy.max_attempts:
35                 logger.error("attempt %d/%d failed: %s; giving up", attempt, policy.max_attempts, exc)
36                 raise RetryExhausted(attempt, exc) from exc
37             wait = policy.delay(attempt)
38             logger.warning("attempt %d/%d failed: %s; retrying in %.0f ms", attempt, policy.max_attempts, exc, wait * 1000)
39             sleep(wait)
```

league_server.py - timeout and retry settings

```
15 # Constants
16 ENV_FILE = Path(__file__).resolve().parents[1] / "api" / ".env" # same DATABASE_URL as the FastAPI app
17 CODE_PATTERN = re.compile(r"FX-\d{5}$") # fixture codes look like FX-00001
18 DB_TIMEOUT = 5 # seconds each attempt may wait for MySQL (connect and read)
19 RETRY_POLICY = RetryPolicy(max_attempts=3, base_delay=0.1, max_delay=1.0)
20 NO_RETRY_CODES = {1044, 1045, 1049} # access denied / unknown database: retrying cannot help
```

league_server.py - timeout on every database attempt

```
self.fault = None # optional FaultInjector used for the Part 3 tests
def query(self, sql: str, params: tuple) -> list[dict]:
    if self.fault:
        self.fault() # may raise TransientError on purpose
    import pymysql # imported here so the server still starts and lists tools without it
    conn = pymysql.connect(**self.config, connect_timeout=DB_TIMEOUT, read_timeout=DB_TIMEOUT,
                           cursorclass=pymysql.cursors.DictCursor)
    try:
        with conn.cursor() as cur:
            cur.execute(sql, params) # %s placeholders, so user input is never pasted into the SQL
            return list(cur.fetchall())
    finally:
        conn.close()
```

league_server.py - retryable errors and the clean error when retries run out

```
def is_retryable(exc: Exception) -> bool:
    """Retry lost connections and timeouts, not mistakes that would fail again."""
    if isinstance(exc, TransientError):
        return True
    if type(exc).__name__ in ("OperationalError", "InterfaceError"): # pymysql connection problems
        return not (exc.args and exc.args[0] in NO_RETRY_CODES)
    return False
def run_query(action) -> dict:
    """Run one store call with retries and wrap any database problem in the envelope."""
    try:
        data, _ = retry_call(lambda: action(get_store()), RETRY_POLICY, is_retryable)
        return ok(data)
    except RetryExhausted as exc:
        return fail("UNAVAILABLE", f"database still failing after {exc.attempts} attempts: {exc.last_error}")
    except ModuleNotFoundError:
        logger.error("pymysql is not installed")
        return fail("DB_ERROR", "pymysql is not installed; start with: mcp dev league_server.py --with pymysql")
    except Exception as exc:
        logger.error("database error: %s", exc)
        return fail("DB_ERROR", f"database request failed: {exc}")
```

Retry demo (retry_demo.py)

```
• (data260) villyalsriprajee@Villyals-MacBook-Pro data260-3319 % python code/mcp_servers/retry_demo.py
2026-10-02 16:06:43,258 INFO league: fixture_details code=FX-00001
2026-10-02 16:06:43,468 INFO league: fixture_details code=FX-00001
2026-10-02 16:06:43,469 WARNING league.retry: attempt 1/3 failed: injected failure (simulated lost database connection); retrying in 100 ms
2026-10-02 16:06:43,603 INFO league: fixture_details code=FX-00001
2026-10-02 16:06:43,603 WARNING league.retry: attempt 1/3 failed: injected failure (simulated lost database connection); retrying in 100 ms
2026-10-02 16:06:43,708 WARNING league.retry: attempt 2/3 failed: injected failure (simulated lost database connection); retrying in 200 ms
2026-10-02 16:06:43,914 ERROR league.retry: attempt 3/3 failed: injected failure (simulated lost database connection); giving up
Retry policy: 3 attempts, backoff 100 ms doubling, max 1000 ms, DB timeout 5 s per attempt

1. Success on the first attempt
  attempts: ok
  time: 210 ms
  result: {"ok": true, "data": {"fixture_code": "FX-00001", "fixture_title": "Pleasanton Hawks vs Cupertino Spartans - Matchweek 32"}, "error": null}

2. Failure on the first attempt, success after a retry
  attempts: fail -> ok
  time: 135 ms
  result: {"ok": true, "data": {"fixture_code": "FX-00001", "fixture_title": "Pleasanton Hawks vs Cupertino Spartans - Matchweek 32"}, "error": null}

3. Failure on every allowed attempt
  attempts: fail -> fail -> fail
  time: 311 ms
  result: {"ok": false, "data": null, "error": {"code": "UNAVAILABLE", "message": "database still failing after 3 attempts: injected failure (simulated lost database connection)"}}
```

Case 1 succeeded on the first attempt. Case 2 failed once, waited 100 ms and succeeded on the retry. Case 3 failed all three attempts and returned a clean UNAVAILABLE envelope after about 300 ms instead of crashing. The output is saved in reports/hw05/raw/p3_retry_demo.txt.

20. Seeded failure simulation

faults.py decides before each database attempt whether it fails, using a random generator seeded with VERIFY_SEED (263319), so the same seed always gives the same success and failure sequence. fault_injection.py runs 50 calls at 0%, 20% and 50% failure across my three tools. Latency includes retries and waits, and p99 is the nearest rank value.

faults.py - seeded fault injection

```
code > mcp_servers > faults.py > FaultInjector
1  # Seeded fault injection: decides before each storage call whether it fails, so the same seed gives the same pattern
2  import random
3  from retry import TransientError
4  VERIFY_SEED = 263319
5  class FaultInjector:
6      """Makes a share of storage calls fail on purpose, reproducibly."""
7      def __init__(self, rate: float = 0.0, seed: int = VERIFY_SEED, plan: list[bool] | None = None):
8          self.rate = rate # chance that one attempt fails
9          self.rng = random.Random(seed) # own generator, so nothing else changes the sequence
10         self.plan = list(plan or []) # fixed outcomes for the demo, used before the random ones
11         self.history: list[bool] = []
12     def __call__(self) -> None:
13         fail = self.plan.pop(0) if self.plan else self.rng.random() < self.rate
14         self.history.append(fail)
15         if fail:
16             raise TransientError("injected failure (simulated lost database connection)")
```

fault_injection.py - running the 150 calls

```
code > mcp_servers > fault_injection.py > main
37 async def main() -> None:
38     store = ls.get_store()
39     tools = {"search_fixtures": ls.search_fixtures, "fixture_details": ls.fixture_details, "tickets_by_city": ls.tickets_by_city}
40     rows, summary = [], []
41     for _ in range(3):
42         await ls.tickets_by_city(5) # warm up so the first timed call does not pay one off start up costs
43     for rate in RATES:
44         store.fault = FaultInjector(rate, VERIFY_SEED) # fresh generator with the same seed for every rate
45         latencies, successes, patterns = [], 0, []
46         for i in range(CALLS_PER_RATE):
47             name, args = pick_call(i)
48             before = len(store.fault.history)
49             start = time.perf_counter()
50             result = await tools[name](**args)
51             latency = (time.perf_counter() - start) * 1000
52             pattern = "".join("F" if f else "S" for f in store.fault.history[before:])
53             successes += result["ok"]
54             latencies.append(latency)
55             patterns.append(pattern)
56             rows.append({"failure_rate": rate, "call": i + 1, "tool": name, "args": json.dumps(args), "ok": result["ok"],
57                         "attempts": len(pattern), "attempt_pattern": pattern,
58                         "error_code": result["error"] if result["error"] else "", "latency_ms": round(latency, 2)})
59         summary.append({"failure_rate": rate, "calls": CALLS_PER_RATE, "success_rate": successes / CALLS_PER_RATE,
60                        "mean_latency_ms": round(sum(latencies) / len(latencies), 2), "p99_latency_ms": round(p99(latencies), 2),
61                        "total_attempts": sum(len(p) for p in patterns),
62                        "sequence_sha256": hashlib.sha256(",".join(patterns).encode()).hexdigest()[:16],
63                        "same_sequence_on_replay": patterns == expected_pattern(rate)})
64     store.fault = None
65     RAW_DIR.mkdir(parents=True, exist_ok=True)
66     with open(RAW_DIR / "p3_fault_injection_calls.csv", "w", newline="") as f:
67         writer = csv.DictWriter(f, fieldnames=list(rows[0]))
68         writer.writeheader()
69         writer.writerows(rows) # all 150 calls
70     (RAW_DIR / "p3_fault_injection_summary.json").write_text(json.dumps(
71         {"verify_seed": VERIFY_SEED, "retry_policy": dataclasses.asdict(ls.RETRY_POLICY), "db_timeout_s": ls.DB_TIMEOUT,
72          "results": summary}, indent=2) + "\n")
73     print(f"VERIFY_SEED={VERIFY_SEED}, {CALLS_PER_RATE} calls per rate, {len(rows)} calls total")
74     print(f"{'Injected failure rate':<23}{'Success rate':<14}{'Mean latency (ms)':<19}{'p99 latency (ms)':<18}{'Attempts':<10}Same sequence")
75     for s in summary:
76         print(f"{'failure_rate':<23.0%}{'success_rate':<14.0%}{'mean_latency_ms':<19.2f}{'p99_latency_ms':<18.2f}"
77               f"{'total_attempts':<10}{'same_sequence_on_replay'}")
78     print(f"Raw data: {RAW_DIR / 'p3_fault_injection_calls.csv'}")
79 if __name__ == "__main__":
80     asyncio.run(main())
```

Fault injection output

VERIFY_SEED=263319, 50 calls per rate, 150 calls total						
Injected failure rate	Success rate	Mean latency (ms)	p99 latency (ms)	Attempts	Same sequence	
0%	100%	26.21	40.88	50	True	
20%	100%	42.61	148.71	57	True	
50%	84%	131.37	365.17	86	True	

Injected failure rate	Success rate	Mean latency (ms)	p99 latency (ms)
0%	100%	26.21	40.88
20%	100%	42.61	148.71
50%	84%	131.37	365.17

"Same sequence: True" confirms the seed reproduces the same pattern. The raw data for all 150 calls is in reports/hw05/raw/p3_fault_injection_calls.csv, with the summary in p3_fault_injection_summary.json.

21. Retry policy evaluation

The policy suits an interactive assistant. At 0% and 20% failure every call succeeded, with a mean under 45 ms and a p99 under 150 ms. At 50% failure, 84% of calls still succeeded with a p99 of about 365 ms, and failed calls returned a clean error after about 300 ms. The one risk is the 5 second timeout: if the database hangs, three attempts could take about 15 seconds, so for interactive use I would lower the timeout to 1 to 2 seconds.

For batch processing, where longer times are acceptable, I would change it as follows:

- Retries: increase to 5 to 8 attempts. At 50% failure, success would rise from about 87.5% to 98% or more.
- Backoff: start at 0.5 to 1 second, double each time, cap at 30 to 60 seconds, and add random jitter.
- Timeout: increase to 15 to 30 seconds so slow but healthy queries are not cut off.

Part 4 - One Safe Tool Entry Point and Offline Tests

22. execute_tool(name, inputs)

execute_tool in tool_entry.py is the one function the Part 5 agent uses to call my three domain tools. It reuses the {ok, data, error} envelope from Part 2B and always returns a JSON string. Before calling a tool it checks the tool name (UNKNOWN_TOOL) and the argument names and types (INVALID_INPUT). Anything unexpected is caught and returned as INTERNAL_ERROR, so it never crashes.

```

code > mcp_servers > tool_entry.py > execute_tool
1 # HW5 Part 4: execute_tool, the single safe entry point the Part 5 agent uses to call the three domain tools (Part 5 adds the safety rule)
2 import asyncio
3 import inspect
4 import json
5 import league_server as ls
6 from envelope import fail
7 TOOLS = {"search_fixtures": ls.search_fixtures, "fixture_details": ls.fixture_details, "tickets_by_city": ls.tickets_by_city}
8 MAX_AGENT_RESULTS = 20 # Part 5 safety rule: the assistant may not pull more than 20 fixtures in one call
9 WILDCARDS = "%"
10 def check_arguments(func, inputs: dict) -> str | None:
11     """Return a problem with the argument names or types, or None when they are fine."""
12     params = inspect.signature(func).parameters
13     unknown = sorted(set(inputs) - set(params))
14     if unknown:
15         return f"unknown argument(s): {' '.join(unknown)}"
16     missing = [n for n, p in params.items() if p.default is inspect.Parameter.empty and n not in inputs]
17     if missing:
18         return f"missing required argument(s): {' '.join(missing)}"
19     for name, value in inputs.items():
20         expected = params[name].annotation
21         if expected in (int, str) and (not isinstance(value, expected) or isinstance(value, bool)):
22             return f"{name} must be {expected.__name__}, got {type(value).__name__}"
23     return None

```

```

32 def execute_tool(name: str, inputs: dict | None = None, store=None) -> str:
33     """Run one domain tool and always return a JSON string in the {ok, data, error} envelope."""
34     previous = ls._store
35     if store is not None:
36         ls._store = store # dependency injection: tests pass a fake store, so no database is needed
37     try:
38         if name not in TOOLS:
39             result = fail("UNKNOWN_TOOL", f"unknown tool '{name}'; available: {' '.join(TOOLS)}")
40         elif not isinstance(inputs if inputs is not None else {}, dict):
41             result = fail("INVALID_INPUT", "inputs must be a JSON object")
42         else:
43             inputs = inputs or {}
44             problem = check_arguments(TOOLS[name], inputs)
45             blocked = None if problem else safety_check(name, inputs)
46             if problem:
47                 result = fail("INVALID_INPUT", problem)
48             elif blocked:
49                 result = fail("SAFETY_BLOCKED", blocked) # rule broken: clean envelope, no exception
50             else:
51                 result = asyncio.run(TOOLS[name](**inputs))
52     except Exception as exc: # last safety net: never crash the caller
53         result = fail("INTERNAL_ERROR", f"{type(exc).__name__}: {exc}")
54     finally:
55         ls._store = previous
56     return json.dumps(result, default=str)

```

23. Test runner

test_runner.py uses plain assert statements. For each tool there is one valid test and one test for the invalid input from Part 3, which makes the six required tests. Four more cover a fixture that does not exist, an unknown tool, a wrong or missing argument, and a storage failure. Every test also checks that the reply is a JSON string with ok, data and error. It prints PASS or FAIL for each test and an X/Y summary at the end. The two Part 5 tests were added to the same runner, so the final run shows 12 tests.

The six required tests

```
27 @test
28 def search_fixtures_valid():
29     r = call("search_fixtures", {"query": "Hawks", "limit": 3})
30     assert r["ok"] and r["data"]["count"] == 2
31     assert [f["fixture_code"] for f in r["data"]["results"]] == ["FX-00001", "FX-00002"]
32 @test
33 def search_fixtures_rejects_short_query():
34     r = call("search_fixtures", {"query": "H", "limit": 3})
35     assert not r["ok"] and r["data"] is None
36     assert r["error"] == {"code": "INVALID_INPUT", "message": "query must be at least 2 characters"}
37 @test
38 def fixture_details_valid():
39     r = call("fixture_details", {"fixture_code": "FX-00001"})
40     assert r["ok"] and r["data"]["team_name"] == "Pleasanton Hawks" and r["data"]["tickets_available"] == 37
41
42
43 @test
44 def fixture_details_rejects_bad_code():
45     r = call("fixture_details", {"fixture_code": "ABC"})
46     assert r["error"] == {"code": "INVALID_INPUT", "message": "fixture_code must look like FX-00001, got 'ABC'"}
47 @test
48 def tickets_by_city_valid():
49     r = call("tickets_by_city", {"top_n": 2})
50     assert r["ok"] and [row["city"] for row in r["data"]] == ["Gilroy", "Santa Clara"]
51     assert r["data"][0] == {"city": "Gilroy", "fixtures": 2, "total_tickets": 411, "avg_tickets": 205.5}
52 @test
53 def tickets_by_city_rejects_zero():
54     r = call("tickets_by_city", {"top_n": 0})
55     assert r["error"] == {"code": "INVALID_INPUT", "message": "top_n must be between 1 and 20, got 0"}
```

Extra tests

```
56 @test
57 def fixture_details_unknown_code_not_found():
58     r = call("fixture_details", {"fixture_code": "FX-99999"})
59     assert r["error"]["code"] == "NOT_FOUND"
60 @test
61 def unknown_tool_is_rejected():
62     r = call("delete_fixture", {"fixture_code": "FX-00001"})
63     assert r["error"]["code"] == "UNKNOWN_TOOL"
64 @test
65 def wrong_argument_type_and_missing_argument():
66     assert call("tickets_by_city", {"top_n": "3"})["error"]["code"] == "INVALID_INPUT"
67     assert call("fixture_details", {})["error"]["message"] == "missing required argument(s): fixture_code"
68 @test
69 def storage_failure_returns_clean_error():
70     store = FakeStore()
71     store.fault = FaultInjector(rate=1.0) # every attempt fails
72     r = call("tickets_by_city", {"top_n": 3}, store=store)
73     assert r["error"]["code"] == "UNAVAILABLE" and len(store.fault.history) == 3 and store.calls == 0
```

PASS/FAIL output and X/Y summary

```
91 def main() -> int:
92     passed = 0
93     for func in TESTS:
94         try:
95             func()
96             passed += 1
97             print(f"PASS {func.__name__}")
98         except AssertionError as exc:
99             reason = str(exc) or traceback.extract_tb(exc.__traceback__)[-1].line # show the failed assert line
100             print(f"FAIL {func.__name__}: {reason}")
101         except Exception as exc:
102             print(f"FAIL {func.__name__}: {type(exc).__name__}: {exc}")
103     print(f"\n{passed}/{len(TESTS)} tests passed")
104     return 0 if passed == len(TESTS) else 1
105 if __name__ == "__main__":
106     sys.exit(main())
```

Test runner output

```
(data260) villyalsriprajee@Villyals-MacBook-Pro data260-3319 % python code/mcp_servers/test_runner.py 2>/dev/null
PASS search_fixtures_valid
PASS search_fixtures_rejects_short_query
PASS fixture_details_valid
PASS fixture_details_rejects_bad_code
PASS tickets_by_city_valid
PASS tickets_by_city_rejects_zero
PASS fixture_details_unknown_code_not_found
PASS unknown_tool_is_rejected
PASS wrong_argument_type_and_missing_argument
PASS storage_failure_returns_clean_error
PASS safety_rule_blocks_bulk_search
PASS run_agent_stops_at_max_steps

12/12 tests passed
```

24. Offline tests

execute_tool takes an optional store, so the tests pass in a FakeStore: 4 fixtures and 3 teams kept in memory, with the same methods as the MySQL store. The runner also points DATABASE_URL at an address that does not exist, so a test that reached the real database would fail. The tests use no MySQL, network or LLM and give the same result every time.

test_runner.py - offline setup and the injected store

```
code > mcp_servers > test_runner.py > main
1 # HW5 Parts 4-5: offline test runner for execute_tool and run_agent (no database, no network, no LLM)
2 import json
3 import os
4 import sys
5 import tempfile
6 from pathlib import Path
7 import traceback
8 os.environ["DATABASE_URL"] = "mysql+pymysql://nobody:nothing@offline.invalid:1/none"
9 import league_server as ls
10 from fake_store import FakeStore
11 from faults import FaultInjector
12 from tool_entry import execute_tool
13 from agent import MockModel, run_agent
14 ls.RETRY_POLICY = ls.RetryPolicy(max_attempts=3, base_delay=0.001, max_delay=0.01) # tiny waits so tests stay fast
15 TESTS = []
16 def test(func):
17     TESTS.append(func)
18     return func
19 def call(name, inputs, store=None):
20     """Run execute_tool with a fresh fake store and check the reply is the JSON envelope."""
21     raw = execute_tool(name, inputs, store=store or FakeStore())
22     assert isinstance(raw, str), "execute_tool must return a JSON string"
23     result = json.loads(raw)
24     assert set(result) == {"ok", "data", "error"}, f"envelope keys wrong: {set(result)}"
25     assert (result["error"] is None) == result["ok"], "error must be null exactly when ok is true"
26     return result
```

fake_store.py - in-memory store

```
17 class FakeStore:
18     """Same three methods as MySQLStore, answered from the small lists above."""
19     def __init__(self, fixtures=None, teams=None):
20         self.fixtures = fixtures if fixtures is not None else FIXTURES
21         self.teams = teams if teams is not None else TEAMS
22         self.fault = None # same hook as MySQLStore, so retry tests can inject failures
23         self.calls = 0 # how many storage calls reached this store
24     def _touch(self):
25         if self.fault:
26             self.fault()
27         self.calls += 1
28     def search_fixtures(self, text: str, limit: int) -> list[dict]:
29         self._touch()
30         text = text.lower()
31         hits = [f for f in self.fixtures if text in f["fixture_title"].lower() or text in f["venue"].lower()]
32         return [{"fixture_code": f["fixture_code"], "fixture_title": f["fixture_title"], "venue": f["venue"],
33                 "tickets_available": f["tickets_available"], "home_team": self.teams[f["home_team_id"]]["name"]}
34                 for f in hits[:limit]]
35     def get_fixture(self, code: str) -> dict | None:
36         self._touch()
37         for f in self.fixtures:
38             if f["fixture_code"] == code:
39                 t = self.teams[f["home_team_id"]]
40                 return {"id": f["id"], "fixture_code": f["fixture_code"], "fixture_title": f["fixture_title"],
41                         "venue": f["venue"], "tickets_available": f["tickets_available"], "created_at": f["created_at"],
42                         "team_id": t["id"], "team_name": t["name"], "team_city": t["city"], "team_email": t["contact_email"]}
43         return None
44     def tickets_by_city(self, top_n: int) -> list[dict]:
45         self._touch()
46         totals = {}
47         for f in self.fixtures:
48             city = self.teams[f["home_team_id"]]["city"]
49             count, total = totals.get(city, (0, 0))
50             totals[city] = (count + 1, total + f["tickets_available"])
51         rows = [{"city": c, "fixtures": n, "total_tickets": s, "avg_tickets": round(s / n, 1)} for c, (n, s) in totals.items()]
52         return sorted(rows, key=lambda r: (-r["total_tickets"], r["city"]))[:top_n]
```

Part 5 - Agent Loop, Safety Rule, and Reflection

1. Safety rule

My rule is that the assistant cannot bulk export fixture data. A search through `execute_tool` can return at most 20 fixtures and cannot use the `%` or `_` wildcards, which would match every fixture. The tool itself allows up to 50, but the agent's entry point is stricter. When the rule is broken, `execute_tool` returns `ok false`, `data null` and a `SAFETY_BLOCKED` error, without raising an exception and before the database is touched.

```
8 MAX_AGENT_RESULTS = 20 # Part 5 safety rule: the assistant may not pull more than 20 fixtures in one call
9 WILDCARDS = "%_"

24 def safety_check(name: str, inputs: dict) -> str | None:
25     """Part 5 business rule: no bulk export of fixture data through the assistant."""
26     if name == "search_fixtures":
27         if inputs.get("limit", 10) > MAX_AGENT_RESULTS:
28             return f"blocked by safety rule: at most {MAX_AGENT_RESULTS} fixtures per search, asked for {inputs['limit']}"
29         if any(ch in inputs.get("query", "") for ch in WILDCARDS):
30             return "blocked by safety rule: wildcard characters (% or _) are not allowed in a search"
31     return None

inputs = inputs or {}
problem = check_arguments(TOOLS[name], inputs)
blocked = None if problem else safety_check(name, inputs)
if problem:
    result = fail("INVALID_INPUT", problem)
elif blocked:
    result = fail("SAFETY_BLOCKED", blocked) # rule broken: clean envelope, no exception
else:
    result = asyncio.run(TOOLS[name](**inputs))
```

Allowed and blocked call (safety_demo.py)

```
• (data260) villyalsriprajeet@Villyals-MacBook-Pro data260-3319 % python code/mcp_servers/safety_demo.py 2>/dev/null
Safety rule: the assistant may get at most 20 fixtures per search and may not use % or _ wildcards

Allowed: search_fixtures({"query": "Gilroy Field", "limit": 5})
-> {"ok": true, "data": "5 fixtures", "error": null}

Blocked: search_fixtures({"query": "Gilroy Field", "limit": 50})
-> {"ok": false, "data": null, "error": {"code": "SAFETY_BLOCKED", "message": "blocked by safety rule: at most 20 fixtures per search, asked for 50"}}

• (data260) villyalsriprajeet@Villyals-MacBook-Pro data260-3319 %
```

Searching "Gilroy Field" with a limit of 5 was allowed and returned 5 fixtures. The same search with a limit of 50 was blocked.

2. Agent loop

`run_agent(user_input)` in `agent.py` sends the question and the three tool descriptions to `qwen2.5:3b` in Ollama. If the model asks for a tool, the call goes through `execute_tool` and the result is sent back to the model. A turn counter stops the loop at `max_steps`. The run ends with one of four stop reasons: `completed`, `safety_block`, `max_steps` or `model_error`. Every start, step, tool call (with its input and result) and the final stop reason is written as one line in `reports/hw05/raw/agent_runs.jsonl`.

agent.py - the Ollama model and the offline MockModel

```
35 class OllamaModel:
36     """Talks to the local Ollama server; temperature 0 and a fixed seed keep runs as repeatable as possible."""
37     def __init__(self, model: str = MODEL_NAME, url: str = OLLAMA_URL, timeout: float = 120.0):
38         self.name, self.url, self.timeout = model, url, timeout
39     def chat(self, messages: list[dict], tools: list[dict]) -> dict:
40         payload = {"model": self.name, "messages": messages, "tools": tools, "stream": False,
41                   "options": {"temperature": 0, "seed": VERIFY_SEED}}
42         response = httpx.post(self.url, json=payload, timeout=self.timeout, trust_env=False) # local server, skip proxies
43         response.raise_for_status()
44         return response.json()["message"] # {"role": "assistant", "content": ..., "tool_calls": [...]}
45 class MockModel:
46     """Offline stand-in for the LLM: returns scripted replies in order, then keeps repeating the last one."""
47     def __init__(self, replies: list[dict]):
48         self.name, self.replies, self.calls = "mock", list(replies), 0
49     def chat(self, messages: list[dict], tools: list[dict]) -> dict:
50         reply = self.replies[min(self.calls, len(self.replies) - 1)]
51         self.calls += 1
52         return reply
```

agent.py - run_agent with the turn counter, max_steps and logging

```
59 def run_agent(user_input: str, model=None, max_steps: int = MAX_STEPS, store=None, log_path: Path = LOG_PATH) -> dict:
60     """Loop: ask the model, run any tool calls through execute_tool, stop on an answer, a safety block or max_steps."""
61     model = model or OllamaModel()
62     run_id = uuid.uuid4().hex[:8]
63     messages = [{"role": "system", "content": SYSTEM_PROMPT}, {"role": "user", "content": user_input}]
64     log_event(log_path, {"run_id": run_id, "event": "start", "model": model.name, "max_steps": max_steps, "user_input": user_input})
65     steps, tool_calls, answer, stop_reason = 0, 0, "", "max_steps"
66     while steps < max_steps:
67         steps += 1 # turn counter
68         start = time.perf_counter()
69         try:
70             reply = model.chat(messages, TOOL_SCHEMAS)
71         except Exception as exc:
72             answer, stop_reason = f"model error: {exc}", "model_error"
73             log_event(log_path, {"run_id": run_id, "event": "model_error", "step": steps, "error": str(exc)})
74             break
75         calls = reply.get("tool_calls") or []
76         log_event(log_path, {"run_id": run_id, "event": "step", "step": steps, "latency_ms": round((time.perf_counter() - start) * 1000),
77                               "content": reply.get("content", ""), "requested_tools": [c["function"]["name"] for c in calls]})
78         if not calls:
79             answer, stop_reason = reply.get("content", ""), "completed" # the model answered without needing a tool
80             break
81         messages.append({"role": "assistant", "content": reply.get("content", ""), "tool_calls": calls})
82         blocked = None
83         for call in calls:
84             name, args = call["function"]["name"], call["function"].get("arguments") or {}
85             if isinstance(args, str):
86                 args = json.loads(args or "{}") # some models send the arguments as a JSON string
87             result = execute_tool(name, args, store=store)
88             tool_calls += 1
89             log_event(log_path, {"run_id": run_id, "event": "tool_call", "step": steps, "tool": name, "input": args, "result": json.loads(result)})
90             messages.append({"role": "tool", "content": result, "tool_name": name})
91             error = json.loads(result).get("error")
92             if error and error["code"] == "SAFETY_BLOCKED":
93                 blocked = error["message"]
94         if blocked:
95             answer, stop_reason = f"I can't do that: {blocked}", "safety_block" # stop the run instead of letting the model retry
96             break
97         if stop_reason == "max_steps":
98             answer = f"stopped: reached max_steps ({max_steps}) before a final answer"
99             log_event(log_path, {"run_id": run_id, "event": "stop", "stop_reason": stop_reason, "steps": steps, "tool_calls": tool_calls, "answer": answer})
100         return {"run_id": run_id, "answer": answer, "stop_reason": stop_reason, "steps": steps, "tool_calls": tool_calls}
101 if __name__ == "__main__":
102     import sys
103     print(json.dumps(run_agent(" ".join(sys.argv[1:]) or "Which 3 cities have the most tickets available?"), indent=2))
```

agent_runs.jsonl - the safety-rule run

```
reports / hw05 > E agent_runs.jsonl
1 ["ts": "2026-10-04T00:18:29.361+00:00", "run_id": "b6d1786d", "event": "step", "step": 1, "latency_ms": 82861, "content": "", "requested_tools": ["search_fixtures"]]
2 ["ts": "2026-10-04T00:18:29.361+00:00", "run_id": "b6d1786d", "event": "tool_call", "step": 1, "tool": "search_fixtures", "input": {"query": "Hawks", "limit": 3}, "result": {"ok": true, "data": {"count": 3, "results": [{"fixture_code": "FX-00001", "fixture_title": "Pleasanton Hawks vs Cupertino Spartans - Matchweek 32 at Pleasanton Sports Complex with 37 tickets left."}]}}]
3 ["ts": "2026-10-04T00:12:18.867+00:00", "run_id": "b6d1786d", "event": "step", "step": 2, "latency_ms": 188902, "content": "I found 3 fixtures for teams called Hawks. Here are the details:\n1. Pleasanton Hawks vs Cupertino Spartans - Matchweek 32 at Pleasanton Sports Complex with 37 tickets left.\n2. Redwood City vs San Jose with 63301 tickets left.\n3. Redwood City vs San Jose with 63301 tickets left.", "requested_tools": []]
4 ["ts": "2026-10-04T00:12:18.867+00:00", "run_id": "b6d1786d", "event": "stop", "stop_reason": "completed", "steps": 2, "tool_calls": 1, "answer": "I found 3 fixtures for teams called Hawks. Here are the details:\n1. Pleasanton Hawks vs Cupertino Spartans - Matchweek 32 at Pleasanton Sports Complex with 37 tickets left.\n2. Redwood City vs San Jose with 63301 tickets left.\n3. Redwood City vs San Jose with 63301 tickets left."}]
5 ["ts": "2026-10-04T00:12:18.867+00:00", "run_id": "18f13a08", "event": "start", "model": "qwen2.5:3b", "max_steps": 5, "user_input": "What is the venue and how many tickets are left for fixture FX-00042?"}]
6 ["ts": "2026-10-04T00:13:40.934+00:00", "run_id": "18f13a08", "event": "step", "step": 1, "latency_ms": 56380, "content": "", "requested_tools": ["fixture_details"]]
7 ["ts": "2026-10-04T00:13:40.934+00:00", "run_id": "18f13a08", "event": "tool_call", "step": 1, "tool": "fixture_details", "input": {"fixture_code": "FX-00042"}, "result": {"ok": true, "data": {"id": 42, "fixture_code": "FX-00042", "fixture_title": "Milpitas Eagles vs San Mateo Mariners - Matchweek 32 at Milpitas Field with 51 tickets left."}}}
8 ["ts": "2026-10-04T00:13:40.934+00:00", "run_id": "18f13a08", "event": "step", "step": 2, "latency_ms": 41389, "content": "The venue for fixture FX-00042 is Milpitas Field and there are 51 tickets left.", "requested_tools": []]
9 ["ts": "2026-10-04T00:13:40.934+00:00", "run_id": "18f13a08", "event": "stop", "stop_reason": "completed", "steps": 2, "tool_calls": 1, "answer": "The venue for fixture FX-00042 is Milpitas Field and there are 51 tickets left."}]
10 ["ts": "2026-10-04T00:13:40.934+00:00", "run_id": "287c11c9", "event": "start", "model": "qwen2.5:3b", "max_steps": 5, "user_input": "Which 3 cities have the most tickets available in total?"}]
11 ["ts": "2026-10-04T00:14:23.097+00:00", "run_id": "287c11c9", "event": "step", "step": 1, "latency_ms": 33155, "content": "", "requested_tools": ["tickets_by_city"]]
12 ["ts": "2026-10-04T00:14:23.097+00:00", "run_id": "287c11c9", "event": "tool_call", "step": 1, "tool": "tickets_by_city", "input": {"stop": 3}, "result": {"ok": true, "data": [{"city": "Gilroy", "fixtures": 250, "total_tickets": 64101}, {"city": "Redwood City", "fixtures": 256, "total_tickets": 63402}, {"city": "San Jose", "fixtures": 256, "total_tickets": 63301}]}]
13 ["ts": "2026-10-04T00:15:27.774+00:00", "run_id": "287c11c9", "event": "step", "step": 2, "latency_ms": 64538, "content": "The 3 cities with the most tickets available are Gilroy with 64101 tickets, Redwood City with 63402 tickets, and San Jose with 63301 tickets.", "requested_tools": []]
14 ["ts": "2026-10-04T00:15:27.774+00:00", "run_id": "287c11c9", "event": "stop", "stop_reason": "completed", "steps": 2, "tool_calls": 1, "answer": "The 3 cities with the most tickets available are Gilroy with 64101 tickets, Redwood City with 63402 tickets, and San Jose with 63301 tickets."}]
15 ["ts": "2026-10-04T00:15:27.774+00:00", "run_id": "28209ff9", "event": "start", "model": "qwen2.5:3b", "max_steps": 5, "user_input": "List 30 fixtures played at Gilroy field."}]
16 ["ts": "2026-10-04T00:16:23.561+00:00", "run_id": "28209ff9", "event": "step", "step": 1, "latency_ms": 56194, "content": "", "requested_tools": ["search_fixtures"]]
17 ["ts": "2026-10-04T00:16:23.561+00:00", "run_id": "28209ff9", "event": "tool_call", "step": 1, "tool": "search_fixtures", "input": {"query": "Gilroy Field", "limit": 30}, "result": {"ok": false, "data": null, "error": {"code": "SAFETY_BLOCKED", "message": "Blocked by safety rule: at most 20 fixtures per search, asked for 30"}}]
18 ["ts": "2026-10-04T00:16:23.561+00:00", "run_id": "28209ff9", "event": "stop", "stop_reason": "safety_block", "steps": 1, "tool_calls": 1, "answer": "I can't do that: blocked by safety rule: at most 20 fixtures per search, asked for 30"}]
19 ["ts": "2026-10-04T00:16:23.561+00:00", "run_id": "47641164", "event": "start", "model": "qwen2.5:3b", "max_steps": 1, "user_input": "Search for Hawks fixtures, then look up the full details of the first one."}]
20 ["ts": "2026-10-04T00:17:04.645+00:00", "run_id": "47641164", "event": "step", "step": 1, "latency_ms": 48602, "content": "", "requested_tools": ["search_fixtures"]]
21 ["ts": "2026-10-04T00:17:04.645+00:00", "run_id": "47641164", "event": "tool_call", "step": 1, "tool": "search_fixtures", "input": {"query": "Hawks"}, "result": {"ok": true, "data": {"count": 18, "results": [{"fixture_code": "FX-00001", "fixture_title": "Pleasanton Hawks vs Cupertino Spartans - Matchweek 32 at Pleasanton Sports Complex with 37 tickets left."}]}}}
22 ["ts": "2026-10-04T00:17:04.645+00:00", "run_id": "47641164", "event": "stop", "stop_reason": "max_steps", "steps": 1, "tool_calls": 1, "answer": "stopped: reached max_steps (1) before a final answer"}]
23
```

3. Additional tests

I added two tests to the Part 4 runner. The first checks that `execute_tool` blocks a search with a limit of 50 before the store is touched, and still allows a limit of 5. The second uses `MockModel`, which keeps asking for a tool and never answers, and checks that `run_agent` stops at `max_steps` = 3 with `stop_reason` `max_steps`. Both run offline with the `FakeStore` and a temporary log file. Both pass in the 12/12 run shown in Part 4.

```
74 @test
75 def safety_rule_blocks_bulk_search():
76     store = FakeStore()
77     r = call("search_fixtures", {"query": "Field", "limit": 50}, store=store) # Part 5 rule: max 20 per search
78     assert r["error"]["code"] == "SAFETY_BLOCKED" and store.calls == 0 # blocked before the store was touched
79     assert call("search_fixtures", {"query": "Field", "limit": 5})["ok"] # the same search within the limit is allowed
80 @test
81 def run_agent_stops_at_max_steps():
82     keeps_calling = {"role": "assistant", "content": "", "tool_calls": [
83         {"function": {"name": "search_fixtures", "arguments": {"query": "Hawks", "limit": 2}}]}}
84     model = MockModel([keeps_calling]) # never gives a final answer
85     with tempfile.TemporaryDirectory() as tmp:
86         log = Path(tmp) / "agent_runs.jsonl"
87         r = run_agent("Find Hawks fixtures", model=model, max_steps=3, store=FakeStore(), log_path=log)
88         events = [json.loads(line) for line in log.read_text().splitlines()]
89         assert r["stop_reason"] == "max_steps" and r["steps"] == 3 and r["tool_calls"] == 3 and model.calls == 3
90         assert events[-1]["event"] == "stop" and events[-1]["stop_reason"] == "max_steps"
```

4. Metrics

I ran five scenarios on `qwen2.5:3b` with `run_scenarios.py`. The results are also in `METRICS.md`.

Scenario output (run_scenarios.py)

```
• (data260) villyalsriprajeet@Villyals-MacBook-Pro data260-3319 % python code/mcp_servers/run_scenarios.py 2>/dev/null
1. Search: steps=2 stop=completed tool_calls=1
   answer: I found 3 fixtures for teams called Hawks. Here are the details:
1. Pleasanton Hawks vs Cupertino Spartans - Matchweek 32 at Pleasanton Sports Complex with 37 t

2. Detail lookup: steps=2 stop=completed tool_calls=1
   answer: The venue for fixture FX-00042 is Milpitas Field and there are 51 tickets left.

3. Aggregate: steps=2 stop=completed tool_calls=1
   answer: The 3 cities with the most tickets available are Gilroy with 64101 tickets, Redwood City with 63402 tickets, and San Jose with 63301 tickets.

4. Safety rule: steps=1 stop=safety_block tool_calls=1
   answer: I can't do that: blocked by safety rule: at most 20 fixtures per search, asked for 50

5. max_steps ceiling: steps=1 stop=max_steps tool_calls=1
   answer: stopped: reached max_steps (1) before a final answer

Model: qwen2.5:3b
| Scenario | Steps | Stop reason | Tool calls |
|-----|-----|-----|-----|
1. Search | 2 | completed | 1 |
2. Detail lookup | 2 | completed | 1 |
3. Aggregate | 2 | completed | 1 |
4. Safety rule | 1 | safety_block | 1 |
5. max_steps ceiling | 1 | max_steps | 1 |

Log: /Users/villyalsriprajeet/Desktop/data260-3319/reports/hw05/agent_runs.jsonl
```

Scenario	Steps	Stop reason	Tool calls
1. Search - Find 3 fixtures for teams called Hawks	2	completed	1
2. Detail - Venue and tickets for FX-00042	2	completed	1
3. Aggregate - Top 3 cities by tickets	2	completed	1
4. Safety rule - List 50 fixtures at Gilroy Field	1	safety_block	1
5. max_steps = 1 - Search, then look up the first one	1	max_steps	1

Scenarios 1 to 3 finished normally: the model called one tool and answered using only its result. Scenario 4 was stopped by the safety rule, and scenario 5 hit the max_steps limit of 1 before it could do the second lookup. The model was slow on my laptop, about 30 to 110 seconds per step.

5. Reflection

For this reflection I picked run 20289ffb from agent_runs.jsonl. The question was "List 50 fixtures played at Gilroy Field."

First the harness logged a start event with the model (qwen2.5:3b), the question and max_steps, which was 5. Then it sent my system prompt, the question and the descriptions of my three tools to Ollama. That counted as step 1.

The model took about a minute to reply. It didn't write any text. It just asked for search_fixtures with the query "Gilroy Field" and a limit of 50. Honestly that was a sensible choice, because the user did ask for 50 fixtures. The harness logged the step and passed the call to execute_tool.

execute_tool checked the arguments first. The names and types were fine, so it moved on to my safety rule, which says the assistant can't pull more than 20 fixtures in one search. 50 is over that, so it returned ok false with a SAFETY_BLOCKED error. It didn't raise an exception, and the database was never queried. The harness logged the tool call with its input and this result.

Then the loop stopped. I made it end the run on a safety block instead of sending the error back to the model, because I didn't want the model to keep trying different ways around the rule. The stop event shows stop_reason safety_block, 1 step and 1 tool call, and the user got a message saying the request wasn't allowed.

So this run ended because of the safety rule, not because it finished or hit max_steps. What I liked about it is that the rule is enforced by my code, so it doesn't depend on the model behaving.

AI use

I used AI as a helper in this homework. It helped me plan each part, explained MCP, retries and the agent loop, and helped me debug setup problems. I implemented the code and then ran and tested it on my own laptop. I ran every command myself, took every screenshot, checked the outputs against my own database, and made the final decisions. All numbers in this report come from my own runs.